

Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Физико-механический институт
Высшая школа прикладной математики и вычислительной физики

Работа допущена к защите

Руководитель ОП

_____ А.М. Левченя

«_____» _____ 2025 г.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
(МАГИСТЕРСКАЯ ДИССЕРТАЦИЯ)

**ОПЫТ ПРИМЕНЕНИЯ НЕЙРОННЫХ ОПЕРАТОРОВ В
ВЫЧИСЛИТЕЛЬНОЙ ГИДРОДИНАМИКЕ**

По направлению подготовки 03.04.01 – Прикладные математика и физика
Направленность (профиль): Инженерно-физические вычисления и машинное обучение

Выполнил:

Трошин М.С.

Студент гр. 5040301/30501

Руководитель:

А.Г. Абрамов

доцент ВШМПивФ ФизМех

кандидат физико-математических наук

Санкт-Петербург

2025

РЕФЕРАТ

На 59 с., 18 рисунков, 5 таблиц, 8 приложений

КЛЮЧЕВЫЕ СЛОВА: ВЫЧИСЛИТЕЛЬНАЯ ГИДРОДИНАМИКА, УРАВНЕНИЯ НАВЬЕ-СТОКСА, ВИХРЬ ТЕЙЛОРА-ГРИНА, НЕЙРОННЫЕ ОПЕРАТОРЫ, FOURIER NEURAL OPERATOR, DEEPONET.

Тема выпускной квалификационной работы: «Опыт применения нейронных операторов в вычислительной гидродинамике».

Данная работа посвящена исследованию, разработке и анализу современных подходов машинного обучения для ускоренного решения задач вычислительной гидродинамики (CFD), нейросетевых операторов в частности. Цель исследования — сравнить современные архитектуры нейронных сетей (PI-DeepONet, PI-TL-DeepONet и Fourier Neural Operator) при прогнозировании эволюции двумерного вихря Тейлора–Грина, описываемой уравнениями Навье–Стокса при высоком числе Рейнольдса.

Основные задачи исследования:

- 1) Обзор существующих подходов машинного обучения к моделированию гидродинамических задач и отбор перспективных нейронных операторов.
- 2) Генерация синтетических наборов данных на основе численного решения в спектральной форме уравнений Навье–Стокса с периодическими граничными условиями.
- 3) Реализация моделей PI-DeepONet, PI-TL-DeepONet и FNO, в которых реализованы физически информированные подходы (physics-informed learning) и обучение с переносом (transfer learning).
- 4) Сравнительный анализ точности (L^2 -ошибка), устойчивости долгосрочного прогноза и ускорения по времени инференса.

Работа выполнена на базе суперкомпьютерного центра СПбПУ; обучение сетей проводилось на GPU NVIDIA V100. Получено, что FNO обеспечивает наилучшую относительную L^2 -погрешность 0,55 на сетке 32×32 при времени инференса ~ 1 с, что делает модель пригодной для расчётов в режиме реального времени. Разработанные рекомендации подтверждают эффективность нейронных операторов как универсального инструмента ускорения численных симуляций без существенной потери точности.

ABSTRACT

59 pages, 18 figures, 5 tables, 8 appendices

KEYWORDS: COMPUTATIONAL FLUID DYNAMICS, NAVIER–STOKES EQUATIONS, TAYLOR–GREEN VORTEX, NEURAL OPERATORS, FOURIER NEURAL OPERATOR, DEEPONET.

Title of the Master’s Thesis: “Application of Neural Operators to Computational Fluid Dynamics”.

This study focuses on the development and analysis of modern machine learning approaches for accelerating computational fluid dynamics (CFD) simulations, with an emphasis on neural operators. The main objective is to compare neural network architectures—PI-DeepONet, PI-TL-DeepONet, and the Fourier Neural Operator (FNO)—in predicting the evolution of a two-dimensional Taylor–Green vortex governed by the Navier–Stokes equations at high Reynolds numbers.

Research objectives:

- 1) Review current machine learning methods for modeling fluid dynamics and select the most promising neural operator architectures.
- 2) Generate synthetic datasets based on spectral-form numerical solutions of the Navier–Stokes equations with periodic boundary conditions.
- 3) Implement the PI-DeepONet, PI-TL-DeepONet, and FNO models incorporating physics-informed learning and transfer learning strategies.
- 4) Conduct a comparative analysis of accuracy (relative L^2 error), long-term prediction stability, and inference-time performance.

The research was conducted using the supercomputing facilities of SPbPU, with model training performed on an NVIDIA V100 GPU. The FNO architecture achieved the best relative L^2 error of 0.55 on a 32×32 grid with an inference time of approximately 1 second, demonstrating the feasibility of using neural operators for real-time simulations. The

formulated recommendations confirm their potential as an efficient tool for accelerating numerical simulations without significant loss of accuracy.

СОДЕРЖАНИЕ

1.	Введение	7
2.	Обзор литературы	8
3.	Разбор архитектуры DeepONet	18
3.1.	Архитектура DeepONet	18
3.2.	Архитектура PI-DeepONet	21
3.3.	Перенос обучения в DeepONet: TL-DeepONet	22
3.4.	Архитектура PI-TL-DeepONet	24
4.	Разбор архитектуры Fourier Neural Operator (FNO)	27
4.1.	Общая идея и постановка задачи	27
4.2.	Архитектура FNO	28
4.3.	Обучение модели FNO	29
4.4.	Примеры применения	29
5.	Постановка задачи моделирования вихря Тейлора–Грина .	31
6.	Генерация данных для обучения DeepONet на задаче вихря Тейлора–Грина	33
6.1.	Начальные условия	33
6.2.	Численное решение уравнений	34
6.3.	Формирование датасета	35
7.	Результаты обучения	39
7.1.	DeepONet	39
7.2.	Fourier Neural Operator (FNO)	44
7.3.	Заключение и выводы по разделу	47
8.	Заключение	49
П1	Генерация данных для задачи вихря Тейлора–Грина	53
П2	DeepONet	55
П3	FNO	58

1. Введение

В последние годы наблюдается стремительное развитие методов машинного обучения, в частности архитектур глубоких нейронных сетей, которое способствует их широкому применению во многих научных и инженерных областях. Особенно перспективным представляется использование этих подходов в вычислительной физике — области, характеризующейся огромным числом задач со сложными многомерными зависимостями — где, в частности, гидродинамические модели традиционно требуют значительных вычислительных ресурсов и временных затрат при численном решении уравнений Навье–Стокса.

В настоящее время методы машинного обучения преимущественно применяются к задачам, для которых накоплены обширные наборы данных. В таких случаях модели обучаются на большом количестве примеров и позволяют получать прогнозы в режиме реального времени без необходимости проведения дорогостоящих численных расчётов.

Однако существует потребность в разработке методов, способных надёжно и универсально справляться с более сложными задачами. Одним из перспективных и актуальных направлений на сегодняшний день являются нейронные операторы.

Целью данной работы является анализ и практическая реализация современных подходов нейронных операторов для решения задачи эволюции вихрей Тейлора–Грина на основе уравнений Навье–Стокса.

2. Обзор литературы

Одной из ключевых работ, положенных в основу литературного обзора, является статья [1], в которой представлен масштабный и систематизированный анализ современных методов машинного обучения, применяемых в области вычислительной гидродинамики (CFD). Данная статья охватывает период 2020–2024 гг. и является наиболее актуальным источником, учитывающим последние достижения в интеграции методов машинного обучения и численного моделирования уравнений в частных производных (PDE).

Авторы структурируют обзор по трём основным направлениям применения методов машинного обучения в вычислительной гидродинамике:

- **Прямое моделирование (forward modeling)** — построение приближённых решений уравнений движения жидкостей и газов с использованием моделей машинного обучения;

- **Инверсные задачи и задачи управления (inverse design & control)** — определение входных параметров (например, начальных или граничных условий) и управляющих воздействий, обеспечивающих достижение целевых характеристик потока;

- **Прикладные области (applications)** — практическое использование подходов с машинным обучением в аэродинамике, горении, климатологии, биомеханике, плазменной физике и других научно-инженерных дисциплинах.

В основу данной работы были выбраны исследования из направления прямого моделирования, в котором авторы выделяют существующие подходы на основе их методологических основ и исследуемых задач, выделяя три ключевых направления:

- **Модели на основе данных (Data-driven surrogates)** — обучаются исключительно на наблюдаемых данных;

- **Физико-информированные модели (Physics-informed surrogate models)** — интегрируют физические знания в обучение;

– Численные методы с элементами машинного обучения (ML-assisted numerical solvers) — частично заменяют традиционные численные методы.

В обсуждаемой статье:

– вводится *новая классификация методов* на основе их мотивации и связи с физическими моделями;

– охватываются как прямые, так и обратные задачи CFD, включая моделирование, проектирование и управление;

– проведён сравнительный анализ более чем 100 архитектур с указанием публикаций, задач и реализаций;

Подробная структура классификации представлена на рисунке 1.

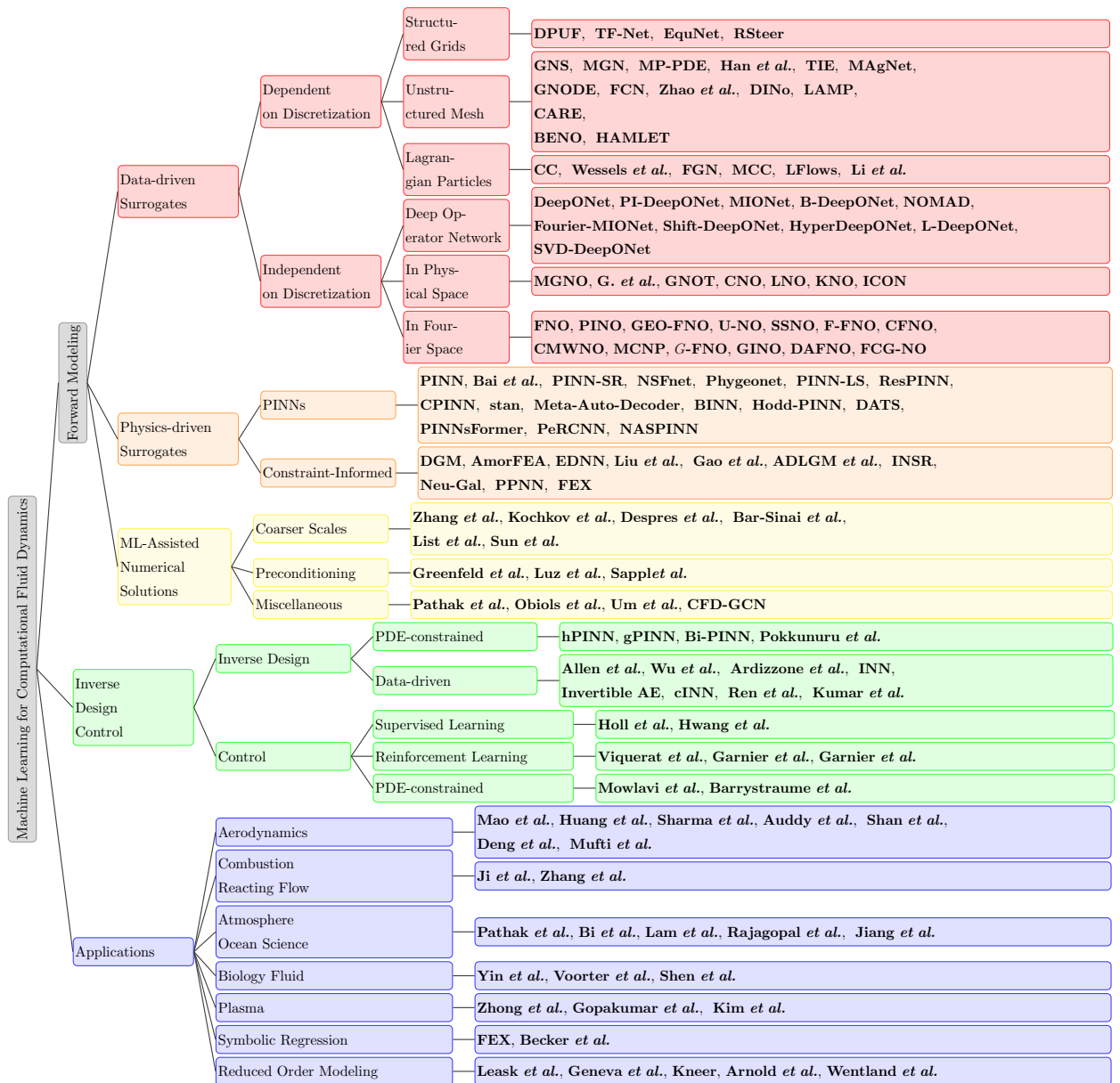


Рисунок 1 – Классификация методов вычислительной гидродинамики (CFD) на основе методов машинного обучения[1].

Обзор [1] представил наиболее полную классификацию методов машинного обучения для гидродинамических задач, отразил текущее состояние области и позволил определить направления дальнейших исследований.

Особый интерес в контексте настоящего обзора представляет работа [2], посвящённая применению методов глубокого обучения для моделирования нестационарного обтекания кругового цилиндра. В данной статье рассматривается задача предсказания пространственно-временной динамики полей скорости и давления, возникающих при раз-

личных числах Рейнольдса. Обучение нейросетевой модели осуществлялось на подмножестве режимов течения, после чего оценивалась её способность к экстраполяции на ранее не представленные в обучающей выборке значения параметров. Такой подход позволяет оценить обобщающую способность модели в условиях изменяющихся режимов обтекания, включая переход от ламинарных к турбулентным структурам течения.

Авторы исследовали четыре типа нейронных сетей:

- CNN (Convolutional Neural Network);
- GAN (Generative Adversarial Network);
- CNN с учетом законов сохранения (Conservation Loss);
- GAN с учетом законов сохранения.

Введение физических функций потерь (loss functions), учитывающих сохранение массы и импульса, позволило повысить физическую правдоподобность предсказаний моделей. В архитектуре GAN генератор предсказывал будущие поля течения на основе предыдущих временных слоев, а дискриминатор проверял их физическую правдоподобность.

Особенности данного подхода:

- прогноз строится на основе четырех предыдущих временных слоев поля течения;
- для обучения использовались результаты численного решения уравнений Навье–Стокса при различных режимах обтекания цилиндра (число Рейнольдса варьировалось в диапазоне $Re = 150...3900$);
- обучение осуществлялось только на части режимов, а тестирование — для режимов с новыми значениями числа Рейнольдса.

Основные результаты:

- все нейросети хорошо справляются с прогнозом одного временного шага вперёд;

- при многократном прогнозировании (10 шагов) ошибки накапливаются, особенно при наличии мелкомасштабных вихрей при больших Re ;
- GAN без физических ограничений показал наименьшие ошибки $L2$ и наиболее стабильное поведение при длительном прогнозе;
- введение физических функций потерь улучшает соблюдение законов сохранения массы и импульса, но увеличивает ошибки приближения.

Работа представляет собой содержательный пример успешного применения методов машинного обучения для моделирования нестационарных течений жидкости и газа.

Особенно интересен подход с использованием GAN и физических функций потерь в силу перспективности для быстрого прогноза больших вихревых структур и возможностей адаптации для задач вычислительной гидродинамики, включая задачи с переменными граничными условиями.

В статье [4] рассматривается применение архитектуры трансформеров (transformers) для моделирования динамики физических систем. Авторы предлагают подход, при котором трансформер используется в качестве суррогатной модели для замены традиционных численных методов решения задач механики, гидродинамики и химической кинетики.

Основная особенность подхода заключается в том, что для перехода от высокоразмерного пространства физических полей (скоростей, давлений, концентраций) к компактному векторному представлению используется специальная эмбединг-модель (embedding model) на основе теории оператора Купмана (Koopman operator). Такой подход позволяет:

- извлекать скрытую динамику системы;
- аппроксимировать её линейным оператором;

– в новом эмбединговом пространстве предсказывать эволюцию системы с помощью трансформера.

Методика предполагает выполнение двух этапов:

1) **Embedding model** — автоэнкодер (autoencoder), обученный выделять Коортан-эмбединги из физических состояний и реконструировать по ним обратно поля системы;

2) **Transformer decoder** — трансформер типа GPT-2, обученный предсказывать Коортан-эмбединги на будущих шагах по времени на основе прошлых.

Во время тестирования предсказанные трансформером эмбединги декодируются обратно в физические состояния.

Основные преимущества подхода:

– трансформер может моделировать сложную многомасштабную динамику (например, переход ламинарного течения в турбулентное);

– способность моделировать распределение начальных условий и параметров, а не только одну конкретную реализацию;

– аппроксимация длинных временных зависимостей благодаря механизму самовнимания (self-attention);

– Коортан-эмбединги позволяют компактно представлять динамику физических систем, сохраняя важную информацию.

Примеры применения архитектур трансформеров в задачах моделирования различных физических систем приведены в таблице 1.

Таблица 1 – Примеры применения архитектур трансформеров

Система	Описание задачи	Результаты
Система Лоренца	Моделирование хаотической системы с множеством начальных условий	Трансформер показал высокую точность и устойчивость к шуму
2D обтекание цилиндра (уравнения Навье–Стокса)	Моделирование перехода от ламинарного к турбулентному режиму течения при разных Re	Трансформер + Koopman embedding смогли воспроизвести детальную структуру вихрей и их периодичность
3D реакционно-диффузионная система Gray–Scott	Сложная многомерная химическая динамика с хаотическими структурами	Трансформер успешно предсказал эволюцию пространственно-временных паттернов

Основные выводы авторов статьи:

- 1) Трансформер + Koopman embedding — это мощный инструмент для построения суррогатных моделей физических систем.
- 2) Трансформер способен эффективно моделировать динамику многовременных процессов без явного указания временной схемы интегрирования.
- 3) Авторы делают акцент на том, что качество эмбединга критически влияет на итоговые результаты (Koopman embedding оказался наиболее удачным).

4) Метод способен заменить традиционные численные методы при многократных прогонках модели (например, в задачах оптимизации и управления).

5) Разработанный трансформер предсказывает динамику систем в сотни раз быстрее традиционных численных методов.

Многомасштабная архитектура PPNN с встраиванием дифференциальных операторов рассматривается в работе [5]. Новый оригинальный подход к построению нейронных сетей для моделирования физических процессов, описываемых уравнениями в частных производных. Основная идея заключается в том, что авторы предлагают не просто добавлять физические ограничения в функцию потерь (как это делается в PINN), а напрямую встраивать дискретизированные PDE-операторы внутрь архитектуры нейронной сети. Такой подход получил название **PDE-Preserved Neural Network (PPNN)**.

В работе показано, что свёртки (convolutions) в нейронных сетях можно интерпретировать как приближения операторов производных (∇ , ∇^2 и т.д.), на основе чего строятся ResNet-блоки, в которых часть остаточной связи (residual connection) напрямую задаётся дискретизированными PDE (например, методом конечных разностей), остальная часть блока остаётся обучаемой и работает как обычная сверточная сеть.

Такой подход позволяет *жестко встроить* известную физику в структуру модели, а неизвестную — дообучить на основе данных.

PPNN архитектурно состоит из двух частей:

- **PDE-preserving block** — встраивает известные члены PDE, реализовано через свёртки с фиксированными ядрами (заданы заранее схемами конечных разностей);

- **Trainable block** — доучивает остальные члены, реализовано через стандартные свёрточные ResNet-блоки.

Между частями реализовано **multi-resolution взаимодействие**:

- PDE-preserving block работает на грубой сетке (обеспечивает стабильность);
- Trainable block — на подробной сетке (обеспечивает точность);
- передача данных между уровнями реализована через *downsampling* и *upsampling*.

Метод протестирован на классических PDE:

- уравнение ФицХью–Нагумо;
- уравнение Бюргерса;
- 2D уравнения Навье–Стокса при различных Re и граничных условиях.

Исследованы разные режимы:

- полностью известные PDE;
- частично известные PDE (например, без источниковых членов);
- неверно заданные PDE (для анализа устойчивости метода).

Показано, что даже при частично известной физике встроенный PDE-блок значительно снижает накопление ошибки при длинном развертывании (rollout) и повышает устойчивость модели на новых параметрах.

В работе [6] представлена простая в использовании открытая библиотека **PINN-2DT**, предназначенная для моделирования нестационарных процессов в двумерной области с помощью метода PINN. Основной акцент сделан на совместимости кода с веб-сервисом Google Colab, что позволяет запускать модели без необходимости использования локальных ресурсов или высокопроизводительных вычислительных кластеров.

В статье рассмотрено четыре примера применения библиотеки:

- модель теплопроводности (heat transfer);
- волновое уравнение (wave equation), моделирующее цунами;
- атмосферная задача с тепловой инверсией (thermal inversion);
- модель роста опухоли (tumor growth model).

Для каждой задачи определены собственные невязки (residuals), исходя из вида конкретного PDE. Продемонстрирована высокая точность восстановления решения и хорошая сходимость по функции потерь.

Библиотека PINN-2DT является полезным инструментом для образовательных целей, а также для быстрого прототипирования моделей, описывающих нестационарные физические процессы в двумерной постановке.

На основе проведенного обзора различных нейросетевых решений в сфере гидродинамики было принято решение рассмотреть и применить нейронные операторы.

3. Разбор архитектуры DeepONet

Исследование нейронных операторов обоснованно начать с одного из первых подходов к их реализации – Deep Operator Networks (DeepONet) [7].

3.1. Архитектура DeepONet

DeepONet (рис. 2) базируется на идее аппроксимации нелинейного оператора, который переводит функцию $u(x)$ из одного функционального пространства в другую функцию и для заданной точки y возвращает численное значение $G(u)(y)$. Основной целью DeepONet является создание эффективного приближения такого оператора.

Простым примером может являться оператор дифференцирования. Если на вход подать функцию $u(x) = x^2$, то оператор вернёт функцию $G(u)(y) = 2x$. DeepONet учится выполнять подобные преобразования для сложных случаев. Архитектура DeepONet представлена на рисунке 2.

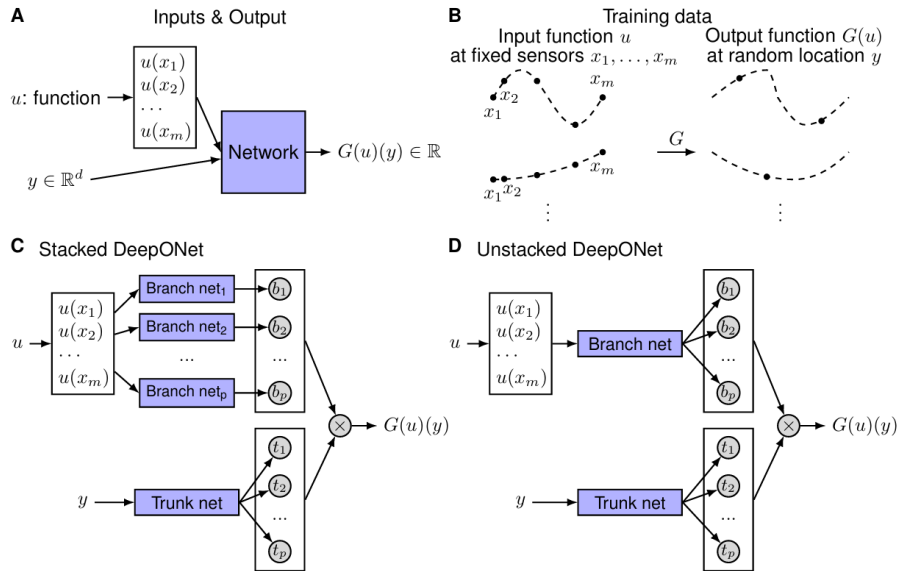


Рисунок 2 – Общая схема архитектуры DeepONet

Структурно DeepONet состоит из двух подсетей. Branch Network (Сеть-ветвь): Обработывает входную функцию $u(x)$. Функция сначала дискретизируется, после чего значения подаются на вход branch network.

Trunk Network (Сеть-ствол): Обрабатывает координаты у тех точек, где необходимо получить значение выходной функции $G(u)(y)$.

Результаты работы обеих подсетей объединяются (обычно через скалярное произведение), что дает финальный результат — значение выходной функции в заданной точке.

Архитектура DeepONet не случайна - она основана на универсальной теореме аппроксимации для операторов, которая математически гарантирует способность такой структуры приближать любой непрерывный оператор.

Теорема 1 (Универсальная теорема аппроксимации для операторов). Пусть σ — непрерывная неполиномиальная функция, X — банахово пространство, $K_1 \subset X$, $K_2 \subset \mathbb{R}^d$ — две компактные области в X и $\mathbb{R}^d$ соответственно, V — компактное множество в $C(K_1)$, G — нелинейный непрерывный оператор, отображающий V в $C(K_2)$.

Тогда для любого $\varepsilon > 0$ существуют положительные целые числа n, p, t и константы $c_i^k, \xi_{ij}^k, \theta_i^k \in \mathbb{R}$, $\zeta_k \in \mathbb{R}$, $w_k \in \mathbb{R}^d$, $x_j \in K_1$, для $i = 1, \dots, n$, $k = 1, \dots, p$, $j = 1, \dots, t$, такие что

$$\left| G(u)(y) - \sum_{k=1}^p \sum_{i=1}^n c_i^k \underbrace{\sigma \left(\sum_{j=1}^t \xi_{ij}^k u(x_j) + \theta_i^k \right)}_{branch} \underbrace{\sigma(w_k \cdot y + \zeta_k)}_{trunk} \right| < \varepsilon \quad (1)$$

для всех $u \in V$ и $y \in K_2$.

Обучение DeepONet производится на множестве пар вида «входная функция – выходная функция» (операторных пар). Входные функции дискретизируются и подаются на вход *branch net*, тогда как соответствующий набор точек решения (дискретизация выходной функции) обрабатывается *trunk net*. Функция потерь формируется на основе известного точного решения, которое может быть получено либо анали-

тически, либо с помощью численного метода, и оптимизируется для минимизации ошибки аппроксимации оператора.

В результате обучения сеть становится способной принимать на вход новые функции, отсутствовавшие в обучающем наборе, и практически мгновенно выдавать приближённое значение выходной функции-оператора. Таким образом, сеть эффективно заменяет трудоёмкие численные расчёты быстрыми прогнозами.

Использование DeepONet позволяет перейти от решения множества отдельных задач (например, задач на уравнение в частных производных с различными начальными или граничными условиями) к решению одной задачи – аппроксимации оператора. Важно отметить, что *branch net* можно многократно комбинировать, и каждая такая подсеть будет генерировать отдельные скаляры, увеличивая выразительную способность сети.

Точка данных в обучающем наборе представляет собой тройку $(u, y, G(u)(y))$. Поскольку для одной и той же входной функции u решение можно оценивать в разных точках y , из одного решения можно получить сразу несколько точек данных. Например, если набор данных содержит 10 000 точек, они могут быть получены из 100 различных входных функций, каждую из которых решают в 100 различных точках.

В контексте вычислительной гидродинамики DeepONet можно использовать следующим образом: при варьировании условий на входе (например, граничные условия и скорость потока) модель обучается предсказывать поля величин (скоростей, давления и т. п.). Потенциально возможно включение в условия задачи и геометрических параметров, что расширит область применимости нейронных операторов в CFD.

3.2. Архитектура PI-DeepONet

Важным развитием архитектуры DeepONet стала модификация *Physics-Informed DeepONet* (PI-DeepONet), предложенная в работе [3]. Этот подход сочетает идею нейронных операторов с методологией *physics-informed learning*, что позволяет учитывать фундаментальные физические законы при обучении модели и существенно снижать потребность в парных обучающих данных вход–выход.

Ключевое отличие PI-DeepONet состоит в том, что обучение производится не только по данным, но и с использованием невязки по уравнению в частных производных (PDE), а также по начальным и граничным условиям. Это особенно актуально в задачах, где получение точных численных решений затруднено или крайне ресурсоёмко.

Общая структура PI-DeepONet сохраняет характерную для DeepONet двойную архитектуру: *branch network* обрабатывает входную функцию $u(x)$ (например, начальные условия), а *trunk network* — координаты y (в пространстве и времени). Их выходы объединяются посредством скалярного произведения для получения предсказания $G(u)(y)$.

Для интеграции физических знаний в процесс обучения функция потерь расширяется следующими компонентами:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{data}} + \lambda_r \mathcal{L}_{\text{residual}} + \lambda_{\text{bc}} \mathcal{L}_{\text{boundary}}, \quad (1)$$

где:

- $\mathcal{L}_{\text{data}}$ — ошибка на известных значениях решения (если они имеются),
- $\mathcal{L}_{\text{residual}}$ — невязка по исходному уравнению в частных производных,
- $\mathcal{L}_{\text{boundary}}$ — невязка граничных условий,
- $\lambda_r, \lambda_{\text{bc}}$ — весовые коэффициенты.

Общая схема PI-DeepONet, включающая физически-информированную компоненту потерь, представлена на рисунке 3.

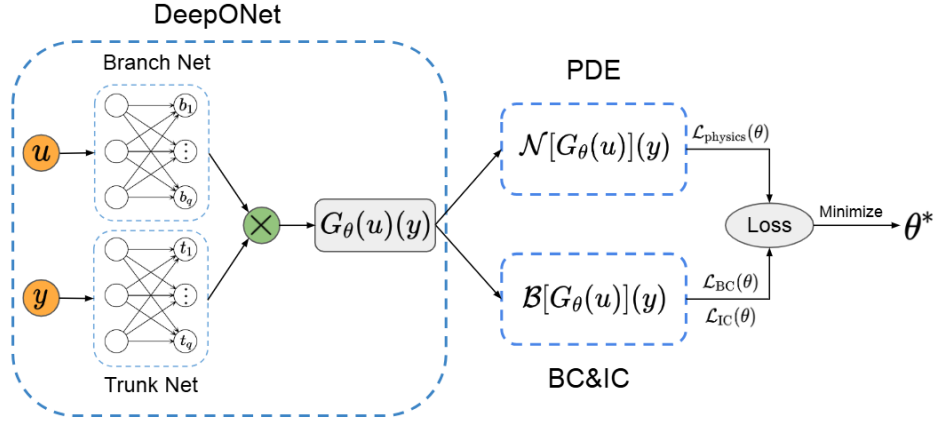


Рисунок 3 – Общая схема архитектуры PI-DeepONet [3].

Благодаря использованию автодифференцирования для вычисления производных, PI-DeepONet легко масштабируется на задачи с произвольной сложностью уравнений. В работе [3] показано, что такая архитектура способна предсказывать решения параметрических PDE за миллисекунды, демонстрируя при этом точность, сопоставимую или превосходящую классические численные методы. Более того, модель успешно работает даже в полностью неаннотированном режиме (unsupervised), когда нет ни одного обучающего примера в виде пар вход-выход, а обучение происходит только на основе формы PDE и граничных условий. Это открывает путь к эффективному решению сложных физических задач в условиях дефицита данных. Функция потерь, использующая невязки по уравнениям, приведена в приложении 5.

3.3. Перенос обучения в DeepONet: TL-DeepONet

TL-DeepONet (Transfer Learning DeepONet) представляет собой расширение базовой архитектуры DeepONet, ориентированное на эффективную адаптацию модели оператора к новой, но связанной задаче. Подход был предложен в работе [10] и направлен на снижение требований к объёму обучающих данных и вычислительным затратам при решении задач с уравнениями в частных производных (PDE).

Когда задача усложняется или изменяются условия (например, геометрия области или параметры течения), переобучение новой DeepONet с нуля может потребовать большого количества данных и вычислительных ресурсов. Подход transfer learning позволяет переиспользовать параметры модели, обученной на исходной (source) задаче, и дообучить её на целевой (target) задаче, обладающей ограниченным числом размеченных примеров. Это особенно актуально в задачах вычислительной гидродинамики, где получение высококачественных численных решений является дорогостоящей процедурой.

Архитектура TL-DeepONet сохраняет ключевые компоненты оригинальной DeepONet: *ветвевую сеть* (branch net), принимающую на вход параметрическую функцию (например, начальное или граничное условие), и *стволовую сеть* (trunk net), обрабатывающую координаты выходной точки y . Модель обучает аппроксимацию оператора $G(u)(y)$ на новом домене, используя предварительно обученную структуру как инициализацию. Схематически процесс переноса обучения в TL-DeepONet и соответствующие модификации архитектуры показаны на рисунке 4.

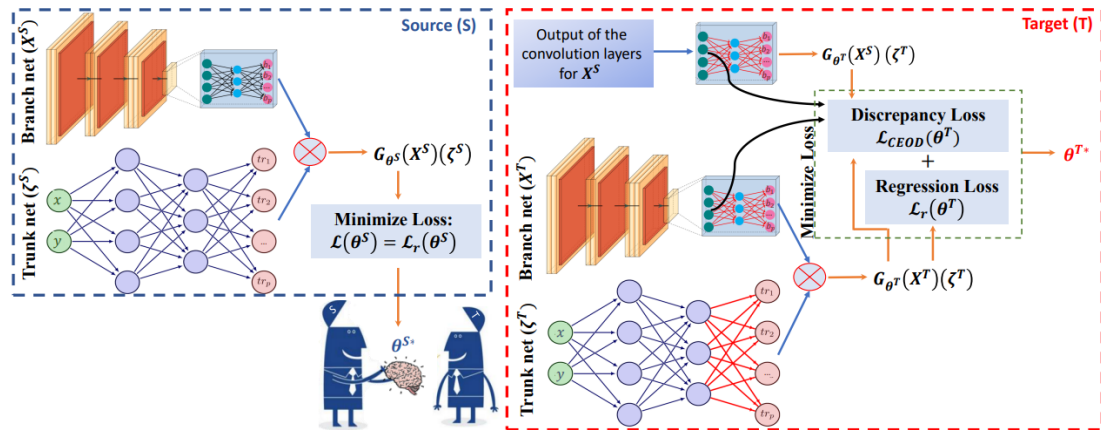


Рисунок 4 – Схема архитектуры TL-DeepONet и процесса переноса обучения.

Ключевая идея TL-DeepONet состоит в фиксации (замораживании) нижних слоёв обеих сетей, ответственных за извлечение обобщённых (домен-независимых) признаков, и в дообучении лишь верхних сло-

ёв: последних полносвязных слоёв ветвевой сети и последнего слоя стволовой сети. Такая стратегия позволяет адаптировать модель к специфике целевой задачи, сохраняя при этом знание, извлечённое из исходной области.

В задачах с временной динамикой (например, реакционно-диффузионные системы), вход модели включает начальное условие и временную координату. TL-DeepONet обучается напрямую на всей временной области, что позволяет моделировать эволюцию состояния во времени без необходимости пошагового прогноза. Такой подход позволяет уменьшить накопление ошибок и обучаться даже при ограниченном числе временных срезов.

Работа [10] демонстрирует, что предложенный подход позволяет достичь высокой точности при малом числе целевых примеров и значительно ускоряет обучение по сравнению с моделями, обучаемыми с нуля.

3.4. Архитектура PI-TL-DeepONet

В недавней работе Wuzhe Xu и соавторов [11] предложен метод **PI-TL-DeepONet** (Physics-Informed Transfer Learning DeepONet), объединяющий архитектуру PI-DeepONet с идеями переноса обучения (transfer learning) для устойчивого долгосрочного прогноза решений эволюционных уравнений.

Базовая идея заключается в том, что решение эволюционного уравнения можно представить как последовательное применение оператора-пропагатора $P_{\Delta t}$:

$$f_{n+1}(x) = P_{\Delta t}(f_n)(x),$$

где f_n — решение в момент времени $n\Delta t$. Модель DeepONet аппроксимирует этот оператор нейросетевым приближением P_{NN} , обучаемым на данных за один временной шаг. Однако при итеративном приме-

нии ошибки аппроксимации накапливаются, что приводит к деградации точности при больших временах.

Для борьбы с этой проблемой авторы предлагают архитектуру PI-TL-DeepONet, ключевыми особенностями которой являются:

– **Физически-информированная функция потерь.** Вместо обучения по данным “вход–выход”, используется самосогласованная функция потерь, основанная на обратной дискретизации по времени:

$$\mathcal{L}(f) = \|P_{\text{NN}}^{-1}(f) - f_{\text{prev}}\|_{L^2}^2 + \|\text{граничные условия}\|^2.$$

Такой подход позволяет обучать модель без необходимости в точных численных решениях, используя только уравнение и начально-граничные условия.

– **Перенос обучения.** При переходе к следующему временному слою фиксируются все параметры сети, кроме последнего слоя ветви (*branch net*), и дообучаются только последние веса w . Это позволяет существенно снизить вычислительные затраты и избежать переобучения.

– **Последовательная адаптация оператора.** Для каждого временного шага n строится новая версия аппроксиматора $P_{\text{NN}}^{(n)}$ за счёт дообучения, и итоговое решение получается как композиция:

$$f_n(x) \approx P_{\text{NN}}^{(n)} \circ P_{\text{NN}}^{(n-1)} \circ \dots \circ P_{\text{NN}}^{(1)}(f_0)(x).$$

На рисунке 5 показана архитектура, PI-TL-DeepONet. Замораживаются все веса, кроме последних в ветви (красная зона).

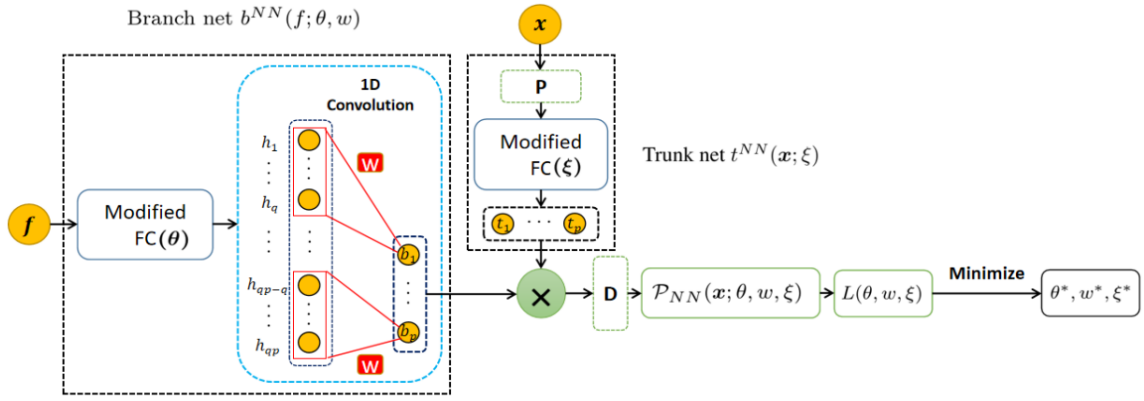


Рисунок 5 – Архитектура PI-TL-DeepONet [11].

Реализация CNN-ветви В модели DeepONet представлена в приложении 4. **Численные эксперименты.** Метод PI-TL-DeepONet протестирован на ряде эволюционных PDE:

- уравнение реакция–диффузия;
- уравнения Аллена–Кана и Кана–Хиллиарда (1D/2D);
- уравнение Навье–Стокса в вихревой формулировке;
- линейное уравнение радиационного переноса с многомасштабными параметрами.

Во всех тестах PI-TL-DeepONet демонстрирует существенно более стабильные и точные долгосрочные предсказания по сравнению с классическим DeepONet. В частности, в задаче Навье–Стокса с вязкостью $\nu = 10^{-3}$ средняя ошибка прогноза на интервале $[0, 1.0]$ уменьшалась с 0.996 (DeepONet) до 0.033 (PI-TL-DeepONet), при увеличении времени вычислений менее чем в 4 раза. Метод также позволяет существенно уменьшить объём обучающих данных при сохранении или улучшении точности.

4. Разбор архитектуры Fourier Neural Operator (FNO)

Нейронные операторы *Fourier Neural Operator* (FNO), предложенные в работе [8], знаменуют собой качественный шаг вперёд в построении сетевых аппроксиматоров операторов. В отличие от DeepONet, который задаёт ядро оператора явной комбинацией двух подсетей, FNO осуществляет нелинейное преобразование целиком в пространстве Фурье, что обеспечивает одновременно *независимость от сетки, квазилинейную сложность алгоритма* и возможность применения модели для предсказания на более тонких сетках, чем те, что использовались при обучении.

Ниже приводится систематический обзор архитектуры, теоретических свойств и практических особенностей FNO.

4.1. Общая идея и постановка задачи

Пусть заданы два банаховых пространства функций $A \subset L^p(D, \mathbb{R}^{d_a})$ и $U \subset L^q(D, \mathbb{R}^{d_u})$ на конечной области $D \subset \mathbb{R}^d$, и пусть нелинейный непрерывный оператор

$$G : A \longrightarrow U, \quad a(\mathbf{x}) \mapsto u(\mathbf{x}) = G(a)(\mathbf{x})$$

задаёт, например, решение параметрического уравнения в частных производных (PDE). Цель FNO — построить приближение $\hat{G} \in \mathcal{N}_\Theta$ (семейство параметризованных сетей) такого, что $\|G(a) - \hat{G}(a)\|_U$ мало для всех входных функций $a \in A$, т. е. обучить *целое семейство PDE* — не один конкретный экземпляр.

Для FNO доказана универсальная теорема аппроксимации операторов (см. [8], Thm. 4.1), аналогичная результату для DeepONet.

4.2. Архитектура FNO

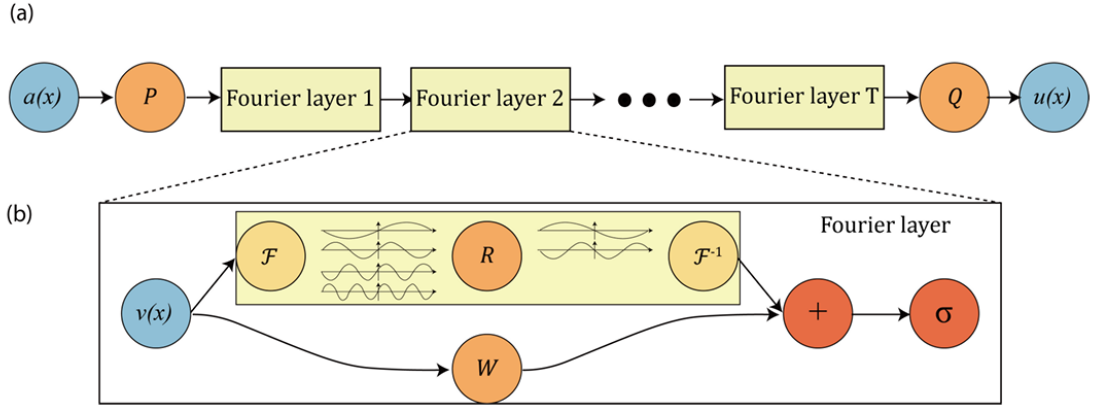


Рисунок 6 – Схема архитектуры Fourier Neural Operator. Блоки FFT/FFT⁻¹ обозначают прямое и обратное преобразования Фурье, R_k — обучаемые комплексные матрицы для низких мод.

Архитектура FNO (рис. 6) состоит из следующих компонентов.

1) **Лифтинг P** : локальная сеть (обычно 1–2 полносвязных слоя) переводит вход $a(\mathbf{x}) \in \mathbb{R}^{d_a}$ в высокоразмерное представление $v^{(0)}(\mathbf{x}) \in \mathbb{R}^{d_v}$.

2) **Чередование T *Fourier-слоёв***: для $t = 0, \dots, T - 1$

$$v^{(t+1)}(\mathbf{x}) = \sigma \left(\underbrace{\mathcal{F}^{-1}(R_{\mathbf{k}}^{(t)} \mathcal{F}[v^{(t)}])}_{\text{глобальный интегральный оператор}} + \underbrace{W^{(t)}v^{(t)}(\mathbf{x})}_{\text{локальный линейный оператор}} \right), \quad (2)$$

где $\mathcal{F}$ и $\mathcal{F}^{-1}$ — прямое и обратное FFT, $R_{\mathbf{k}}^{(t)} \in \mathbb{C}^{d_v \times d_v}$ — обучаемые матрицы, заданные *только для K низких пространственных частот* ($|\mathbf{k}| \leq k_{\max}$), остальные моды обнуляются. Нелинейность $\sigma(\cdot)$ применяется покомпонентно (обычно ReLU или GELU). Таким образом слой реализует свёрточный интегральный оператор $(\mathcal{K}^{(t)}v^{(t)})(\mathbf{x})$ с ядром, заданным *неявно* в Фурье-области.

3) **Проекция Q** : заключительный линейный (или неглубокий нелинейный) блок понижает размерность $v^{(T)}(\mathbf{x}) \mapsto \hat{u}(\mathbf{x}) \in \mathbb{R}^{d_u}$.

Реализация комплексного спектрального умножения дана в приложении 6, а соответствующий модуль спектральной свёртки — в при-

ложении 7. Архитектура базового блока FNO с четырьмя спектральными слоями и пропускной связью приведена в приложении 8.

Ключевые свойства.

– *Инвариантность к дискретизации.* Параметры $R_k^{(t)}$ обучаются в спектральном пространстве и потому не зависят от конкретной сетки; FNO можно обучить на грубой решётке n^d и применять на более тонкой без дообучения (*zero-shot super-resolution*).

– *Квазилинейная сложность.* Стоимость одного слоя — $O(n^d \log n^d + K d_v^2)$, где первая часть — FFT, вторая — матричное умножение на ограниченном числе мод $K \ll n^d$.

– *Экспрессия высоких частот.* Ограничение по $k_{\max}$ не обрезает окончательное решение: нелинейности σ и декодер Q восстанавливают высокочастотные компоненты, что подтверждается спектральным анализом.

4.3. Обучение модели FNO

Датасет. Точка данных имеет форму $(a, \{\mathbf{x}_i\}, u_i)$, где a — входная функция (параметр PDE), а $u_i = G(a)(\mathbf{x}_i)$ — значения решения в узлах сетки. Как и у DeepONet, из одного решения можно сформировать множество точек за счёт различного субдискретизирования по $\mathbf{x}$.

Функция потерь. Обычно используется L_2 -среднеквадратичная ошибка $\mathcal{L}_{\text{data}} = \frac{1}{N} \sum_{j=1}^N \|\hat{G}(a_j) - u_j\|_2^2$. Для временных PDE возможна физически информированная надбавка (аналогично PI-DeepONet): $\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{data}} + \lambda_r \mathcal{L}_{\text{residual}} + \lambda_{\text{bc}} \mathcal{L}_{\text{boundary}}$.

Оптимизация. Исходная работа использует Adam с learning-rate decay; регуляризация по спектральному радиусу матриц R_k помогает стабилизировать обучение при высоких режимах Рейнольдса.

4.4. Примеры применения

Модель FNO успешно применялась к различным PDE-задачам:

– **Уравнение Бюргерса (1D):** ошибка предсказания $< 1\%$ при обучении на 1000 примерах.

– **Дарси-флоу (2D):** превосходство по точности над DeepONet и стандартными CNN.

– **Уравнения Навье–Стокса (2D, турбулентный режим):** FNO обеспечивает стабильное обучение при $\nu = 10^{-4}$, демонстрируя zero-shot super-resolution при переходе от 64×64 к 256×256 .

На практике время инференса модели на GPU составляет миллисекунды, что делает FNO особенно привлекательной для задач инверсного проектирования и ускоренной симуляции.

5. Постановка задачи моделирования вихря Тейлора–Грина

Одним из классических тестов для проверки корректности численных методов решения уравнений Навье–Стокса является задача моделирования вихря Тейлора–Грина (Taylor–Green vortex). Данная задача представляет собой аналитически заданное начальное условие с периодическими граничными условиями, эволюция которого приводит к развитой вихревой структуре, затухающей за счёт вязкости.

Рассматривается двумерная несжимаемая вязкая жидкость, описываемая уравнением Навье–Стокса в форме для завихренности:

$$\frac{\partial \omega}{\partial t} + \mathbf{u} \cdot \nabla \omega = \nu \nabla^2 \omega, \quad (3)$$

где:

- $\omega(x, y, t)$ — завихренность потока,
- $\mathbf{u} = (u, v)$ — поле скорости, связанное с завихренностью через $\omega = \partial_x v - \partial_y u$,
- $\nu = \frac{1}{\text{Re}}$ — кинематическая вязкость (обратная числу Рейнольдса).

Вычисление начальной завихренности

Пусть

$$\mathbf{u}(x, y, 0) = (u(x, y, 0), v(x, y, 0)) = (-\cos(\pi x) \sin(\pi y), \sin(\pi x) \cos(\pi y)).$$

Тогда завихренность определяется как

$$\omega = \nabla \times \mathbf{u} = \frac{\partial v}{\partial x} - \frac{\partial u}{\partial y}.$$

Для наших начальных условий получаем:

$$\begin{aligned} \frac{\partial v}{\partial x}(x, y, 0) &= \frac{\partial}{\partial x} [\sin(\pi x) \cos(\pi y)] = \pi \cos(\pi x) \cos(\pi y), \\ \frac{\partial u}{\partial y}(x, y, 0) &= \frac{\partial}{\partial y} [-\cos(\pi x) \sin(\pi y)] = -\pi \cos(\pi x) \cos(\pi y), \end{aligned}$$

откуда

$$\omega(x, y, 0) = \pi \cos(\pi x) \cos(\pi y) - (-\pi \cos(\pi x) \cos(\pi y)) = 2\pi \cos(\pi x) \cos(\pi y).$$

Граничные условия на границе области $\Omega = [0, 1]^2$ считаются периодическими по x и y (см. схему на рисунке 7).

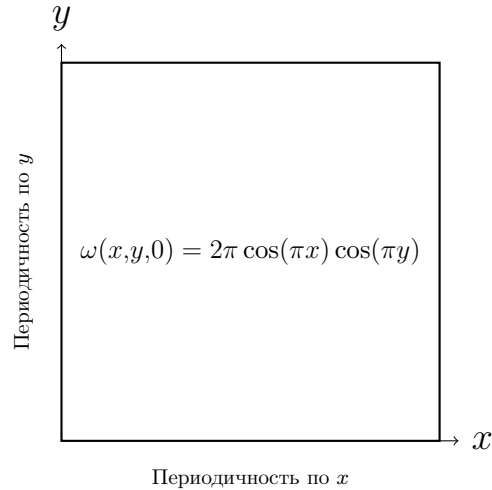


Рисунок 7 – Схема постановки задачи вихря Тейлора–Грина: расчетная область $[0, 1]^2$ м, периодические граничные условия и начальное условие.

Цель задачи

Целью численного моделирования является построение решения $\omega(x, y, t)$ и/или $\mathbf{u}(x, y, t)$ на интервале времени $t \in [0, T]$.

Параметры задачи

Для моделирования используются следующие параметры:

- Размер расчётной области: $[0, 1] \times [0, 1]$ м;
- Тип граничных условий: периодические;
- Вязкость: $\nu = 10^{-4}$ м²/с (в зависимости от режима);
- Временной интервал: $t \in [0, T]$, где T выбирается в зависимости от устойчивости схемы;
- Размер сетки: $N \times N$, где $N = 16, 32$

6. Генерация данных для обучения DeepONet на задаче вихря Тейлора–Грина

В данной работе генерируются синтетические данные на основе двумерного вихря Тейлора–Грина (Taylor–Green Vortex) для обучения нейросетевых моделей. Математической моделью выступает уравнение Навье–Стокса в завихренностной формулировке с периодическими граничными условиями:

$$\frac{\partial \omega}{\partial t} + \mathbf{u} \cdot \nabla \omega = \nu \nabla^2 \omega, \quad (4)$$

где ω — завихренность, $\mathbf{u} = (u, v)$ — поле скорости, а $\nu = 1/\text{Re}$ — кинематическая вязкость. Скорость восстанавливается из потенциала тока ψ через:

$$u = \frac{\partial \psi}{\partial y}, \quad v = -\frac{\partial \psi}{\partial x}, \quad \nabla^2 \psi = -\omega. \quad (5)$$

6.1. Начальные условия

В качестве начальных условий используется аналитическое задание с параметрами случайной амплитуды и фазы:

$$\omega_0(x, y) = 2\pi A \sin(2\pi(x + \varphi_x)) \sin(2\pi(y + \varphi_y)) + \text{шум}, \quad (6)$$

где $A \sim \mathcal{U}(a, b)$, $\varphi_x, \varphi_y \sim \mathcal{U}(0, 1)$. Шум вычитается по среднему, чтобы $\int \omega_0 dx dy = 0$.

Программный код для задания начальных условий:

```
1 A = torch.empty(batch, 1, 1).uniform_(0.8, 1.2)
2 phx = torch.rand(batch, 1, 1)
3 phy = torch.rand(batch, 1, 1)
4 base = 2 * math.pi * A * torch.sin(2 * math.pi * (X + phx)) *
    torch.sin(2 * math.pi * (Y + phy))
```

6.2. Численное решение уравнений

Решение проводится в частотной области с помощью спектрального метода Фурье и схемы Кранка–Николсона. Производные и операторы Лапласа вычисляются как:

$$\mathcal{F}[\nabla^2 f] = -4\pi^2(k_x^2 + k_y^2)\hat{f}, \quad (7)$$

$$\mathcal{F}\left[\frac{\partial f}{\partial x}\right] = ik_x\hat{f}, \quad \mathcal{F}\left[\frac{\partial f}{\partial y}\right] = ik_y\hat{f}. \quad (8)$$

Пример кода для решения уравнения Пуассона:

```
1 lap = 4*(math.pi**2)*(k_x**2 + k_y**2)
2 psi_h[...,0] = w_h[...,0] / lap
3 psi_h[...,1] = w_h[...,1] / lap
```

Скорости:

```
1
2 q[...,0] = -2 * math.pi * k_y * psi_h[...,1]
3 q[...,1] = 2 * math.pi * k_y * psi_h[...,0]
4
5
6 v[...,0] = 2 * math.pi * k_x * psi_h[...,1]
7 v[...,1] = -2 * math.pi * k_x * psi_h[...,0]
```

Нелинейный член рассчитывается в физическом пространстве и обратно преобразуется в спектральное:

$$F_h = \mathcal{F}\left[u\frac{\partial\omega}{\partial x} + v\frac{\partial\omega}{\partial y}\right] \quad (9)$$

```
1 F_h = torch.rfft(q * w_x + v * w_y, 2, normalized=False,
    onesided=False)
```

Схема Кранка–Николсона:

$$\omega_h^{n+1} = \frac{-\Delta t F_h^n + \Delta t f_h^n + (1 - 0.5\Delta t\nu L)\omega_h^n}{1 + 0.5\Delta t\nu L} \quad (10)$$

Код:

```
1 w_h[... , 0] = (-dt * F_h[... , 0] + dt * f_h[... , 0] + (1 - 0.5 *  
    dt * visc * lap) * w_h[... , 0]) / (1 + 0.5 * dt * visc * lap)
```

Подробный программный код реализации численного шага решения уравнений Навье–Стокса приведён в приложении 1, 2.

6.3. Формирование датасета

Для каждой траектории сохраняется эволюция $\omega(x, y, t_k)$, $\psi_x(x, y, t_k)$, $\psi_y(x, y, t_k)$. Все поля преобразуются к векторному виду и сохраняются как входы и выходы для DeepONet:

- $\text{Train_B} \in \mathbb{R}^{N_{\text{train}} \cdot M \times N^2}$ — значения ω ,
- $\text{Train_psi_x}, \text{Train_psi_y}$ — поля скоростей,
- $\text{Test_evo_mat} \in \mathbb{R}^{N_{\text{test}} \times M \times N^2}$ — тестовые эволюции.

Формат сохранения:

```
1 np.savez("ns_data_deeponet.npz",  
2         ic_train=my_ic_train, sol_train=my_sol_train,  
3         ic_test=my_ic_test, sol_test=my_sol_test)
```

Пример генерации полей

Примеры эволюции поля завихренности для различных значений вязкости приведены на рисунках 8 и 9.

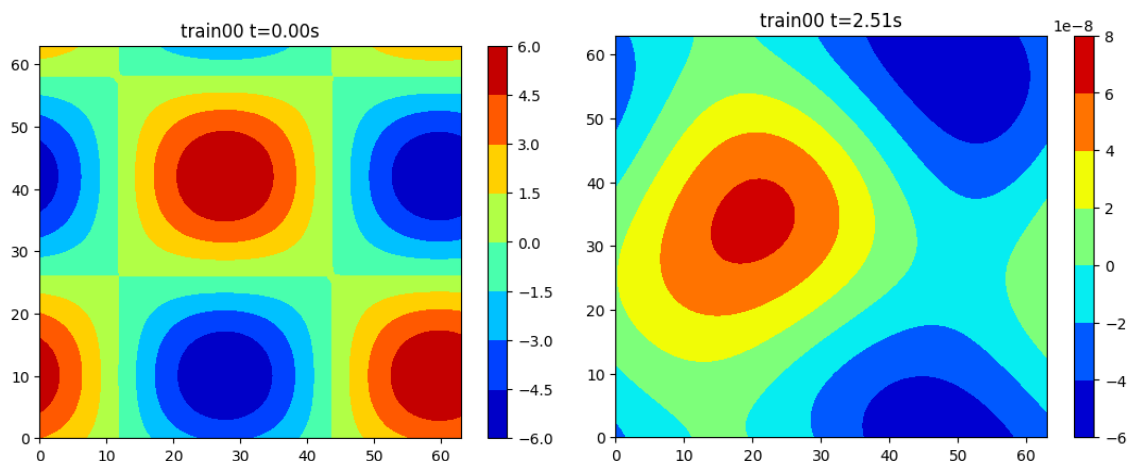


Рисунок 8 – Поле завихренности $\omega(x, y, t)$ на начальном моменте времени $t = 0$ с (слева) и в момент времени $t = 2.5$ с (справа). Видно, как структура вихря эволюционирует и затухает из-за вязкости $\nu = 0.1 \text{ м}^2/\text{с}$.

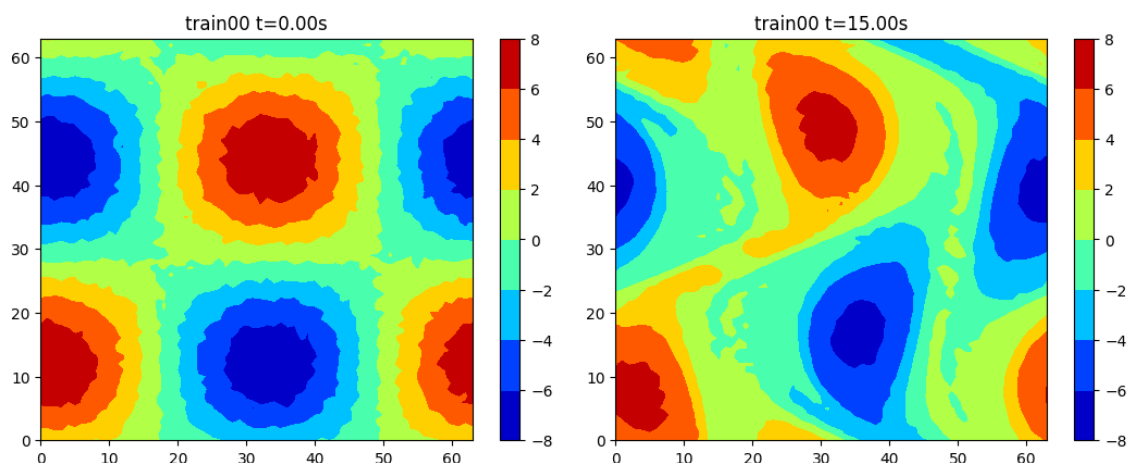


Рисунок 9 – Поле завихренности $\omega(x, y, t)$ на начальном моменте времени $t = 0$ с (слева) и в момент времени $t = 15.0$ с (справа). Видно, как структура вихря эволюционирует и затухает из-за вязкости $\nu = 10^{-4} \text{ м}^2/\text{с}$.

Сравнение результатов с ANSYS Fluent

С целью валидации реализованного численного метода было выполнено сравнение его результатов с эталонным решением, полученным в пакете ANSYS Fluent. Постановка задачи во Fluent включала двумерное течение несжимаемого воздуха с вязкостью $\nu = 10^{-4} \text{ м}^2/\text{с}$ и плотностью $\rho = 1 \text{ кг}/\text{м}^3$ в области $[0, 1]^2$ с периодическими граничными

условиями. Начальное поле скорости было задано в виде:

$$u(x,y,0) = 0.5 \cdot \sin(2\pi x) \cdot \cos(2\pi y),$$

$$v(x,y,0) = -0.5 \cdot \cos(2\pi x) \cdot \sin(2\pi y),$$

что соответствует начальному завихренному полю $\omega_0 = 2\pi \cdot \sin(2\pi x) \cdot \sin(2\pi y)$. Расчёт в Fluent проводился на сетке 64×64 с временем моделирования $T = 100$ с и шагом записи 0.5 с. Завихренность вычислялась по формуле $\omega_z = \partial_x v - \partial_y u$ в центрах ячеек.

График сравнения рассчитанной завихренности по координате (0.5, 0.5) приведён на рисунке 10.

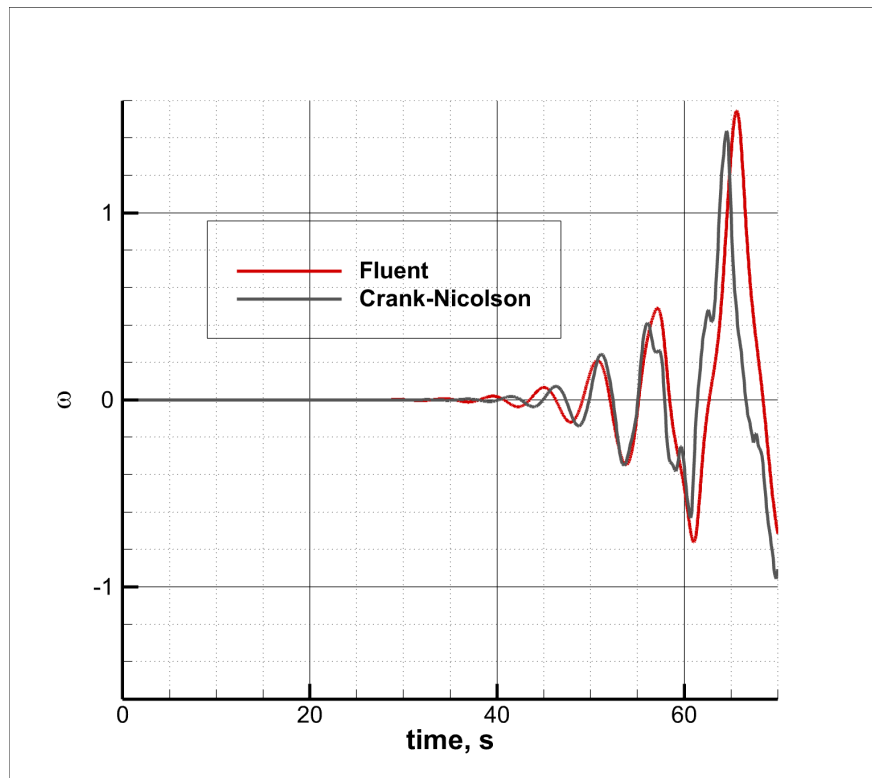


Рисунок 10 – Сравнение полученной завихренности в зависимости от времени в точке (0.5, 0.5).

Для оценки качества реализованного численного решения с использованием спектрального метода и схемы Кранка–Николсона была проведена количественная сверка с результатами Fluent. В таблице 2 представлены значения средней ошибки, стандартного отклонения и относительной ошибки по величине завихренности ω в трёх контрольных

точках области за первые 70 секунд моделирования. Количественные значения ошибок представлены в таблице 2.

Таблица 2 – Сравнение завихренности ω между Fluent и численным решением (первые 70 секунд)

Точка	Средняя ошибка	Ст. отклонение	Отн. ошибка, %
(0.50, 0.50)	0.102	0.211	10.14
(0.80, 0.80)	0.537	0.253	14.15
(0.20, 0.20)	0.310	0.282	8.18

Полученные результаты демонстрируют удовлетворительное согласие между спектральным численным решением и эталонной CFD-симуляцией. Несмотря на заметное расхождение в амплитудах в отдельных точках (особенно в (0.50, 0.50)), основной интерес представляет именно динамика: как видно из графика на рис. 10, распад вихрей происходит синхронно как по времени, так и по форме кривых. Это подтверждает корректность численного метода и позволяет использовать его для генерации достоверных данных в задачах обучения нейронных операторов.

7. Результаты обучения

Результаты, представленные в настоящем разделе, получены с использованием вычислительных ресурсов суперкомпьютерного центра Санкт-Петербургского политехнического университета Петра Великого <https://www.spbstu.ru>. Обучение всех рассмотренных моделей проводилось на графическом ускорителе **NVIDIA V100** (32 GB).

7.1. DeepONet

Для количественной оценки способности нейронных операторов аппроксимировать эволюцию вихря Тейлора–Грина выполнено сравнение архитектур *PI-DeepONet*, *PI-TL-DeepONet* и *FNO* при двух пространственных разрешениях ($N = 16, 32$) и фиксированной кинематической вязкости $\nu = 10^{-4} \text{ м}^2/\text{с}$.

Качество прогноза контролировалось тремя взаимодополняющими метриками:

1) средняя относительная погрешность в норме L^2 , вычисляемая во времени

$$\text{Rel } L^2(t) = \frac{\|\omega_{\text{pred}} - \omega_{\text{ref}}\|_{L^2}}{\|\omega_{\text{ref}}\|_{L^2}}; \quad (11)$$

2) эмпирическая функция потерь на обучающей выборке;

3) визуальное сопоставление мгновенных полей завихренности $\omega(x, y, t)$ в контрольные моменты времени.

Параметры модели и вычислительные затраты

Использовалась архитектура с параметрами:

– **Оптимизатор:** Adam с экспоненциальным спадом скорости обучения от $0.25 \cdot 10^{-3}$ до 10^{-5} ;

– **Размер батча:** 128;

– **Размерность сети:**

– branch basis size = 100,

– trunk basis size = 60,

– число слоёв trunk-net = 7,

– число нейронов на слой = 256;

- **Сеточные разрешения:** 16×16 , 32×32 ;
- **Число обучающих примеров:** 200;
- **Число тестовых примеров:** 50.
- **Промежуток эволюции для обучения:** 15 с.
- **Промежуток эволюции для теста:** 25 с.

Время обучения одной модели представлено в таблице 3.

Таблица 3 – Время обучения модели DeepONet на разных разрешениях

Разрешение сетки	Время обучения	Число эпох
16×16	~6 часов	10 000
32×32	~8 часов	10 000

Увеличение вычислительных затрат при переходе к сетке 32×32 объясняется квадратичным ростом размерности входного тензора branch-сети, а также повышенным объёмом операций автоматического дифференцирования.

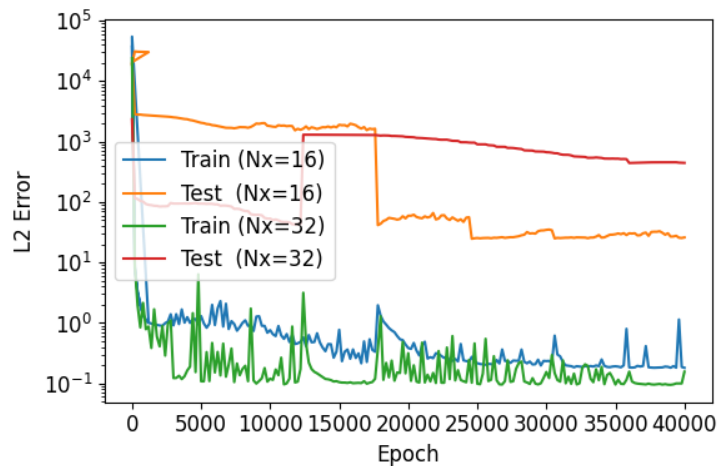


Рисунок 11 – Ошибка L2 на тренировочных и тестовых данных во время обучения моделей на сетке 16×16 и 32×32 .

На логарифмическом графике ошибок (рис. 11) наблюдается монотонное уменьшение $\text{Rel } L^2$ до стационарного уровня $\approx 10^{-1}$ ($N_x = 32$)

после $6,5 \times 10^3$ эпох. Периодические всплески на тренировочной кривой обусловлены взаимодействием шага оптимизации с экспоненциальным распадом скорости обучения.

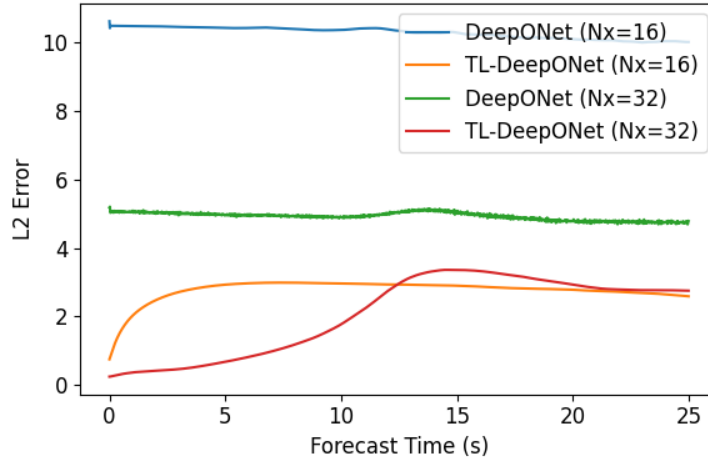


Рисунок 12 – Ошибка L2 относительно времени прогноза для сеток 16x16 и 32x32.

Из графика $\text{Rel } L^2(t)$ на горизонте $T = 25\text{с}$ (рис. 12) следует, что PI-TL-DeepONet удерживает ошибку на низком уровне лучше, чем PI-DeepONet и увеличивается по мере эволюции вихрей. Базовая PI-DeepONet демонстрирует почти постоянную ошибку на всем графике эволюции.

Базовая PI-DeepONet демонстрирует почти стационарную ошибку $\text{Rel } L^2 \approx 5$ ($N_x = 32$) и ≈ 10 ($N_x = 16$), что подчёркивает влияние временного линейного уточнения [7].

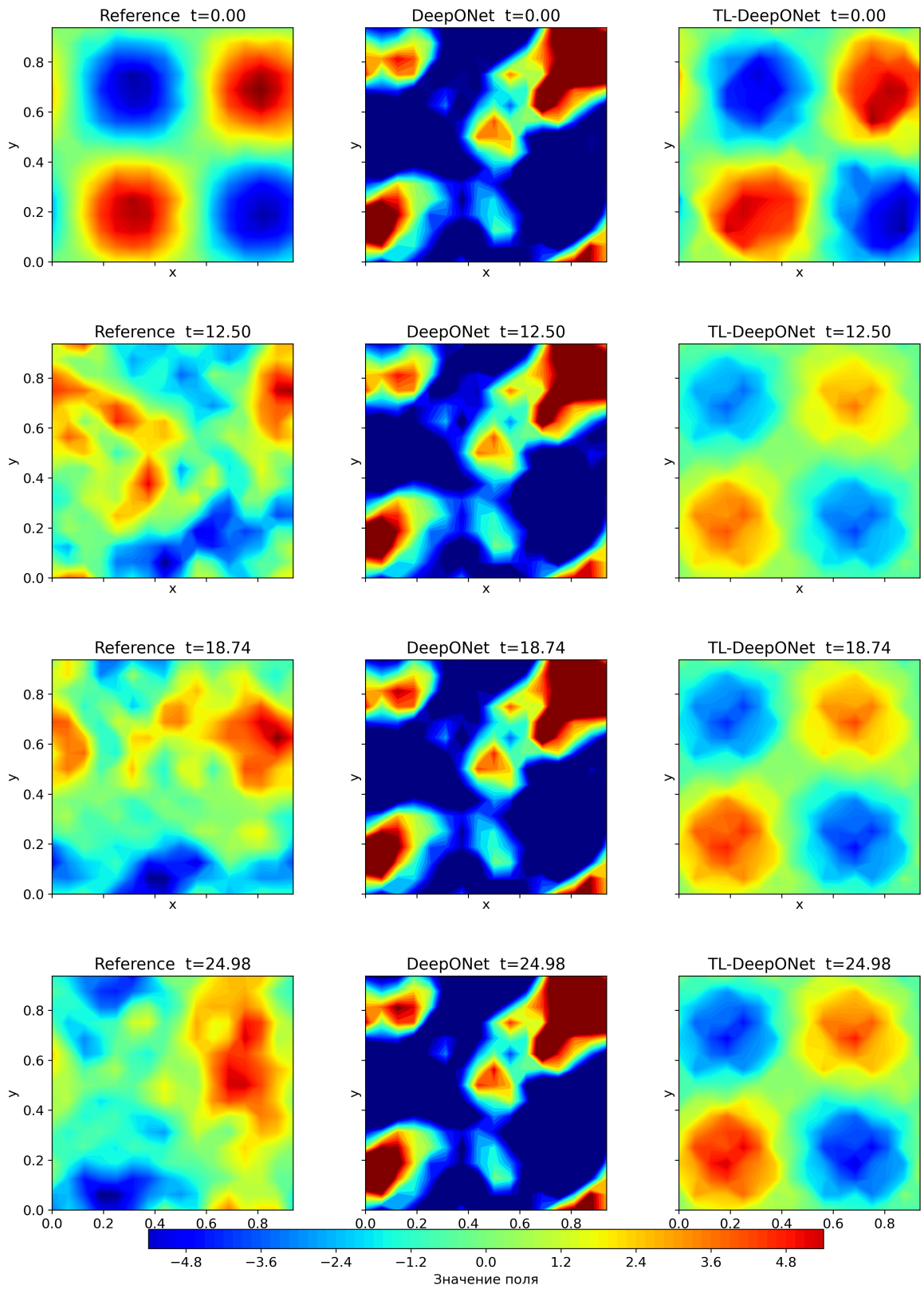


Рисунок 13 – Поле завихренности в разные моменты времени для архитектуры PI-DeepONet и PI-TL-DeepONet, сетки 16x16 при вязкости $\nu = 10^{-4} \text{ м}^2/\text{с}$.

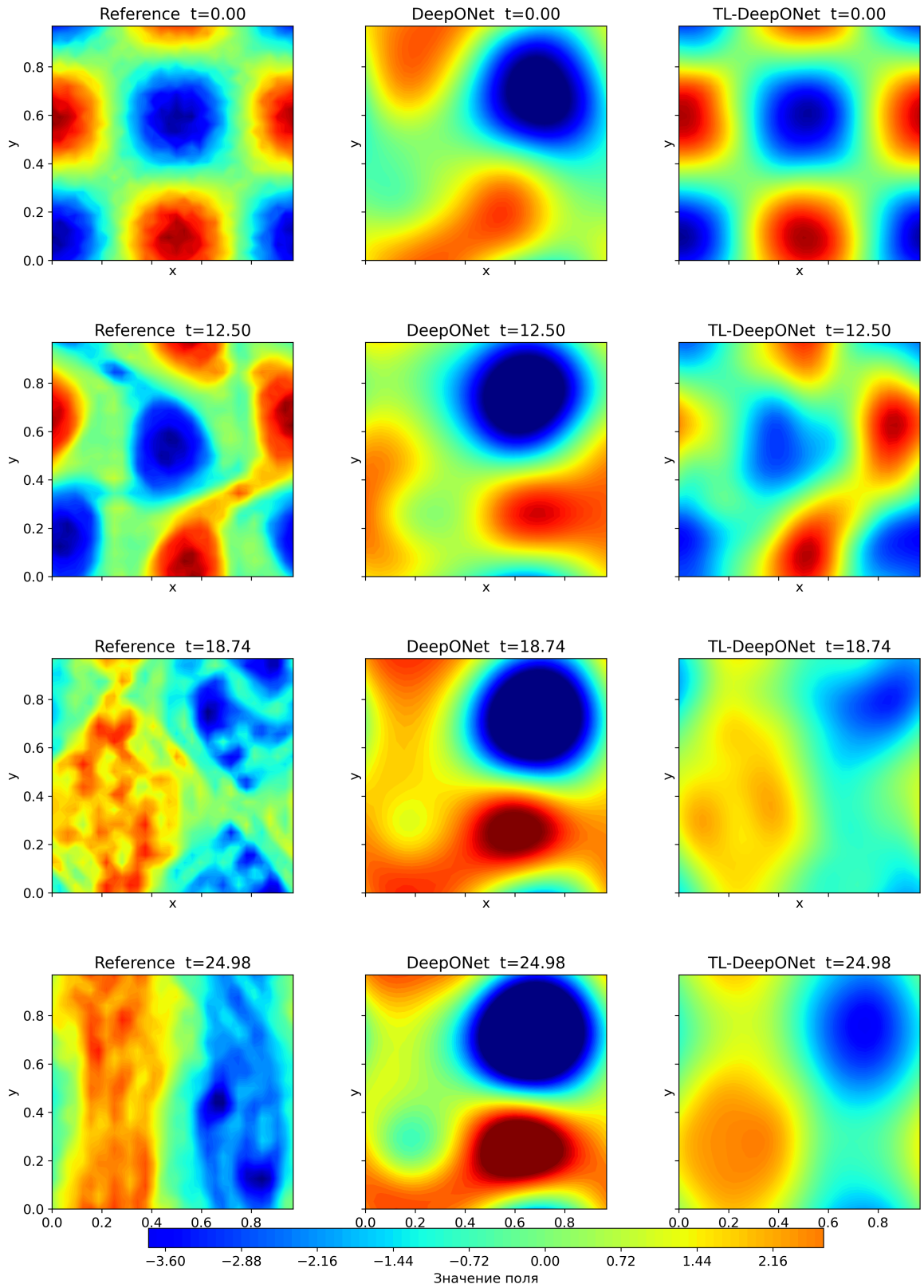


Рисунок 14 – Поле завихренности в разные моменты времени для архитектуры PI-DeepONet и PI-TL-DeepONet, сетки 32×32 при вязкости $\nu = 10^{-4} \text{ м}^2/\text{с}$.

В контрольные моменты времени $t/T_{\max} = \{0, 0.5, 0.75, 1\}$ (рис. 13, 14) PI-TL-DeepONet корректно воспроизводит крупномас-

штабные структуры при 32×32 , тогда как для 16×16 наблюдаются заметные отклонения спектра мелких вихрей.

7.2. Fourier Neural Operator (FNO)

Для оценки качества аппроксимации оператора эволюции вихря Тейлора–Грина проведено сравнение архитектуры Fourier Neural Operator (FNO) при двух пространственных разрешениях ($N = 16, 32$) и вязкости $\nu = 10^{-4}$.

Использовались те же три взаимодополняющие метрики:

- 1) средняя относительная погрешность в норме L^2 во времени
- 2) эмпирическая функция потерь на обучающей выборке;
- 3) визуальное сопоставление мгновенных полей завихренности $\omega(x, y, t)$ в контрольные моменты времени.

Параметры модели и вычислительные затраты

Реализация FNO выполнена на **PyTorch**. Гиперпараметры:

- **Оптимизатор:** Adam, начальный $\text{lr} = 2.5 \times 10^{-3}$, weight decay 10^{-4} ;
- **Шаг обучения:** каждые 100 эпох $\gamma = 0.5$;
- **Число эпох:** 1 000;
- **Размер батча:** 10;
- **Параметры FNO:**
 - число Фурье-модов на каждом измерении $m_1 = m_2 = 8$,
 - ширина скрытой части $\text{width} = 20$;
- **Сеточные разрешения:** 16×16 , 32×32 ;
- **Число обучающих примеров:** 200;
- **Число тестовых примеров:** 50.
- **Промежуток эволюции для обучения:** 15 с.
- **Промежуток эволюции для теста:** 25 с.

Время обучения одного запуска:

Таблица 4 – Время обучения FNO на разных разрешениях (1000 эпох)

Разрешение сетки	Время обучения	Число эпох
16×16	30 ч	1 000
32×32	30 ч	1 000

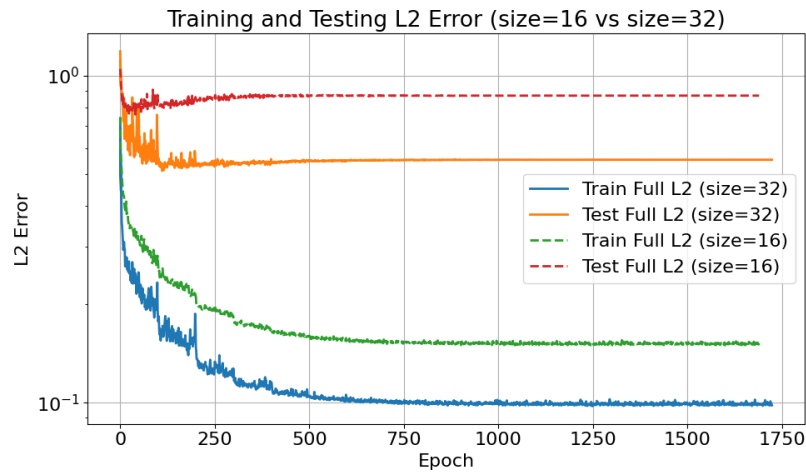


Рисунок 15 – Ошибка L2 на тренировочных и тестовых данных во время обучения моделей на сетке 16x16 и 32x32.

Кривые на рис. 15 демонстрируют экспоненциальный спад ошибки до значений $\text{Rel } L_{\text{test}}^2 \approx 0,25$ ($N_x = 32$) и $\approx 0,6$ ($N_x = 16$) после ~ 700 эпох, но далее ошибка продолжает уменьшаться с меньшей скоростью. Отсутствие выбросов свидетельствует о стабильности спектральных слоёв и меньшей чувствительности к размеру шага обучения по сравнению с PI-TL-DeepONet.

Экспоненциальный спад тестовой ошибки до $\text{Rel } L^2 \approx 0.25$ ($N_x = 32$) достигается уже к 700-й эпохе (рис.15); последующее обучение приводит к медленному снижению ошибки.

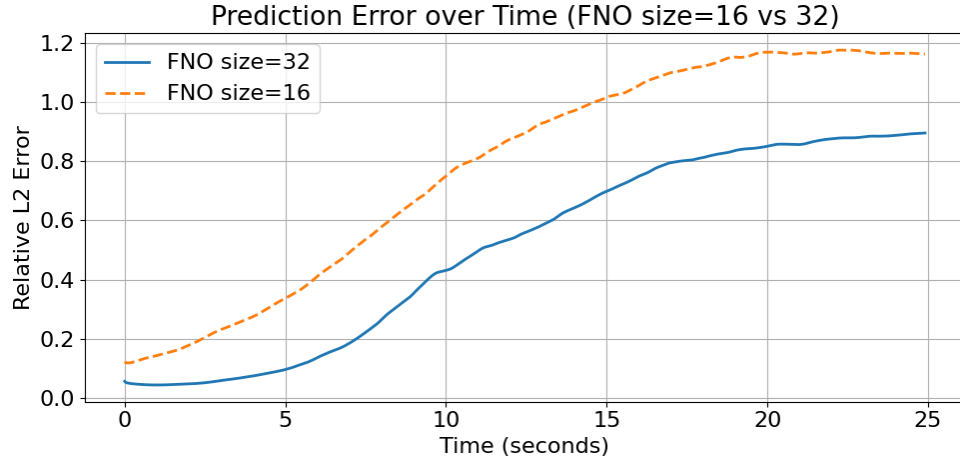


Рисунок 16 – Ошибка L2 относительно времени прогноза для сеток 16x16 и 32x32.

Кривые $\text{Rel } L^2(t)$ (рис.16) растут почти линейно; более пологий наклон при $N = 32$ подтверждает, что FNO эффективно масштабируется с числом степеней свободы.

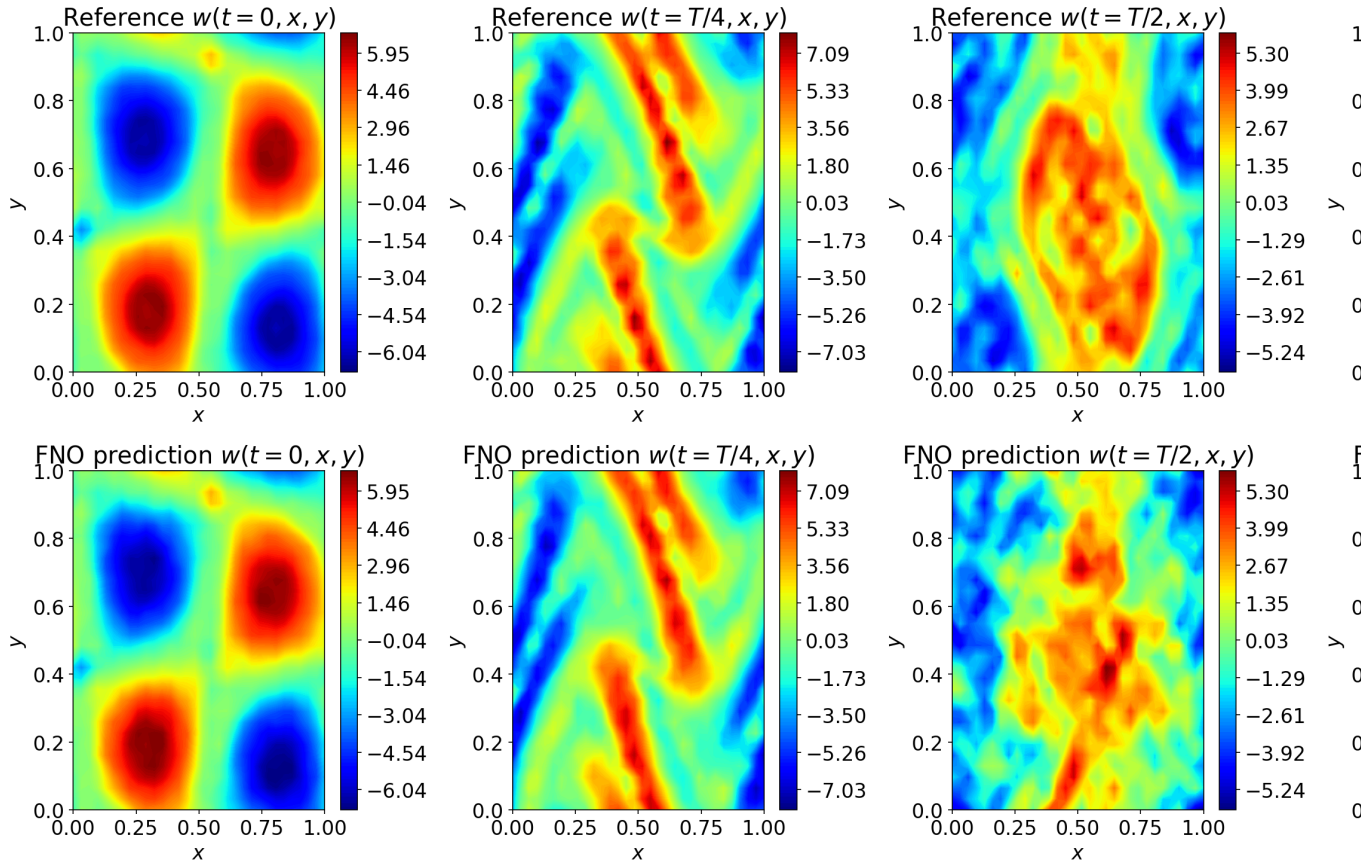


Рисунок 17 – Поле завихренности в разные момент времени для архитектуры FNO, сетки 32x32 при вязкости $\nu = 10^{-4} \text{ м}^2/\text{с}$.

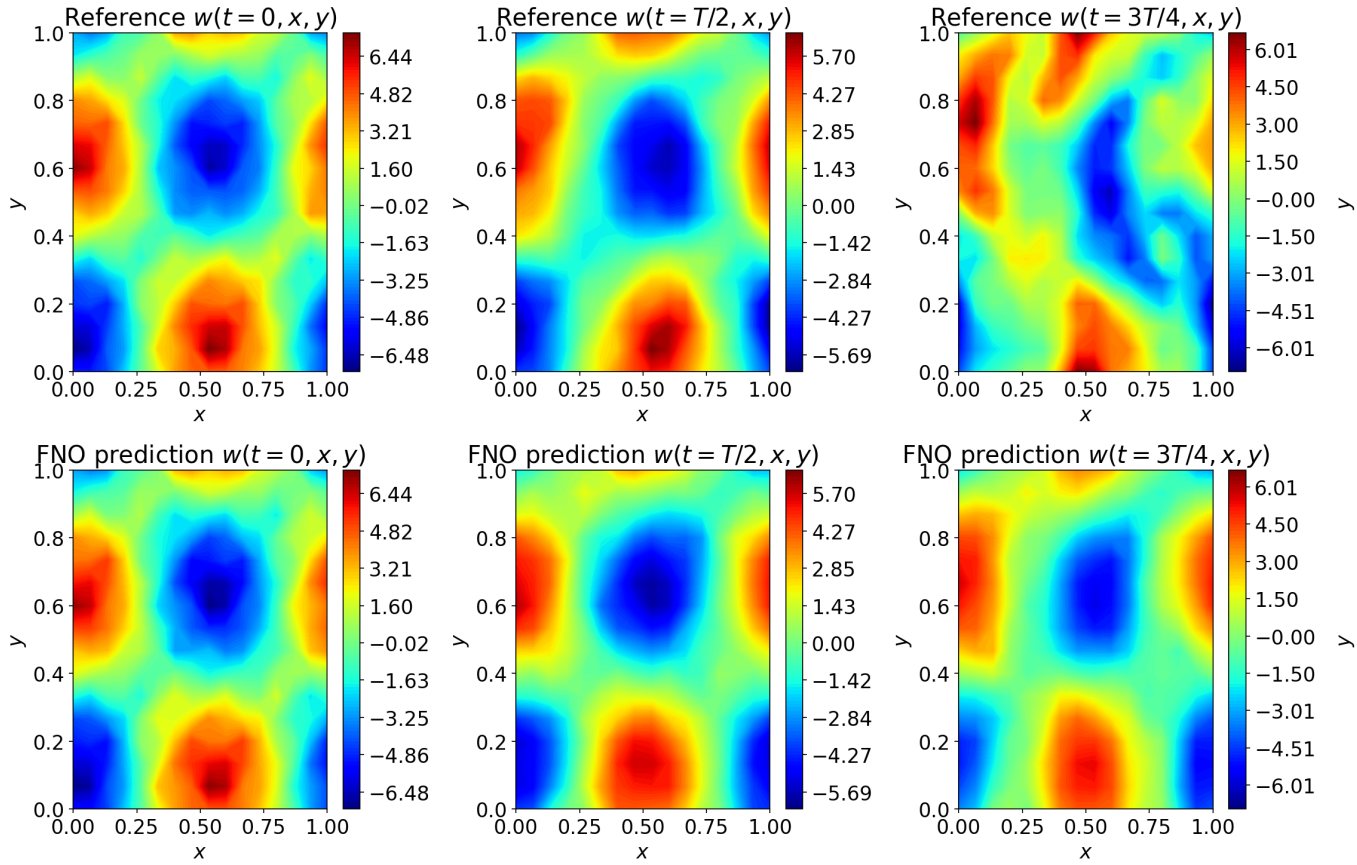


Рисунок 18 – Поле завихренности в разные момент времени для архитектуры FNO, сетки 16x16 при вязкости $\nu = 10^{-4}\text{м}^2/\text{с}$.

На рисунках 17–18 FNO видно, что для данных с сеткой 32x32 архитектура обучается воспроизводить поле завихренности на начальных этапах. Но дисперсия увеличивается со временем, хотя общие очертания остаются схожими. Для данных с сеткой 16x16 мы также наблюдаем сильные расхождения в прогнозируемых полях.

7.3. Заключение и выводы по разделу

Сравнительные значения средних относительных ошибок моделей представлены в таблице 5.

Таблица 5 – Сравнение относительных L^2 -ошибок моделей на разных разрешениях

N_x	FNO	PIDON	TL-PIDON
32	0.5527	1.7070	0.8469
16	0.8679	3.3400	0.9124

Сопоставление средних относительных L^2 -ошибок (табл. 5) позволяет сделать следующие выводы.

1) **Наиболее точной моделью оказалась FNO.** При дискретизации 32×32 FNO обеспечивает минимальную ошибку $\text{Rel } L^2 = 0.55$, что на 35% лучше, чем результат PI-TL-DeerONet ($\text{Rel } L^2 = 0.85$) при сопоставимом объёме вычислений.

2) **Обычная PI-DeerONet неконкурентоспособна.** Ошибка базовой архитектуры превышает 1.7 (для 32×32) и 3.3 (для 16×16), что делает её непригодной для детального прогноза динамики вихря.

3) **Сетка 16×16 недостаточна для всех моделей.** Переход к огрублённой дискретизации приводит к росту ошибки FNO до 0.87 и PI-TL-DeerONet до 0.91, а поле завихренности воспроизводится лишь частично. Таким образом, разрешение $N = 16$ не обеспечивает требуемой точности описания эволюции вихря Тейлора–Грина.

4) **Промежуточный вывод для дальнейших исследований.** При фиксированном горизонте прогноза $T = 25$ s и сопоставимом времени обучения именно FNO демонстрирует наилучший баланс между сохранением крупномасштабных структур и контролем суммарной ошибки.

8. Заключение

В настоящем исследовании поставлена задача оценки потенциала нейронных операторов при моделировании двумерных течений вязкой несжимаемой жидкости, описываемых уравнениями Навье–Стокса. Работа имела преимущественно исследовательский характер и была направлена на выявление достоинств и ограничений современных архитектур машинного обучения в вычислительной гидродинамике. Основные результаты можно сформулировать следующим образом.

1) **Разработана и протестирована универсальная методика генерации синтетических данных** для вихря Тейлора–Грина. Применение спектрально-точной схемы Кранка–Николсона позволило формировать контролируемые выборки разного разрешения и вязкости, обеспечивая воспроизводимость численных экспериментов.

2) **Выполнен углублённый аналитический разбор и практическая реализация трёх нейронных операторов** — *PI-DeepONet*, *PI-TL-DeepONet* и *Fourier Neural Operator (FNO)*. Для PI-моделей использованы физически-информированные функции потерь, а в PI-TL-DeepONet протестирована стратегия переноса обучения, позволившая сократить время сходимости и повысить устойчивость прогноза.

3) **Проведён сравнительный численный эксперимент** на двух сеточных разрешениях (16×16 и 32×32). Наилучшие средние показатели относительной ошибки L^2 достигнуты FNO (например, $\text{Rel } L^2 = 0.5527$ при $N = 32$), тогда как перенос обучения улучшил базовую DeepONet и обеспечил стабильность прогноза на горизонте $T = 25 \text{ s}$. Полученные значения служат ориентиром для будущих исследований, но не претендуют на окончательное решение задачи.

Задача заключалась в *исследовании* возможностей нейронных операторов, выявлении их достоинств и границ применимости в контексте гидродинамических задач. Полученные результаты демонстрируют перспективы таких моделей, однако окончательные выводы о целесо-

образности их использования в промышленной CFD требуют дальнейшего глубокого анализа.

Перспективы дальнейших исследований

- Расширение постановки на трёхмерные течения и сложные геометрии, включая нестационарные граничные условия.
- Изучение поведения нейронных операторов при широком диапазоне чисел Рейнольдса и переменных физических параметрах.
- Интеграция графовых и адаптивно-сеточных архитектур для нерегулярных доменов.
- Комбинация нейронных операторов с традиционными численными методами в гибридных вычислительных схемах.

Таким образом, работа внесла вклад в систематизацию знаний о нейронных операторах в вычислительной гидродинамике и предоставила исходные данные для их дальнейшего совершенствования и практического применения.

Список литературы

1. Wang, H., Cao, Y., Huang, Z., et al. *Recent Advances on Machine Learning for Computational Fluid Dynamics: A Survey*. arXiv:2408.12171, 2023.
2. Lee, S., You, D., & others. *Data-driven prediction of unsteady flow over a circular cylinder using deep learning*. arXiv:1804.06076, 2018.
3. Lu, L., Jin, P., Pang, G., Zhang, Z., & Karniadakis, G.E. *Learning the solution operator of parametric partial differential equations with physics-informed DeepONets*. arXiv:2103.10974, 2021.
4. Zhong, Y., Tallec, C., Leclerc, A., & Bietti, A. *Transformers for Modeling Physical Systems*. arXiv:2010.03957, 2020.
5. Liu, X.-Y., Zhu, M., Lu, L., et al. *Multi-resolution partial differential equations preserved learning framework for spatiotemporal dynamics*. arXiv:2205.03990, 2022.
6. Cao, S., Wu, W., & Karniadakis, G.E. *Physics-Informed Neural Network Code for 2D Transient Problems (PINN-2DT)*. arXiv:2310.03755, 2023.
7. Lu, L., Jin, P., & Karniadakis, G.E. *DeepONet: Learning nonlinear operators for identifying differential equations based on the universal approximation theorem of operators*. arXiv:1910.03193, 2021.
8. Li, Z., Kovachki, N., Azizzadenesheli, K., Liu, B., Bhattacharya, K., Stuart, A., & Anandkumar, A. *Fourier Neural Operator for Parametric Partial Differential Equations*. In International Conference on Learning Representations (ICLR), 2021. arXiv:2010.08895.
9. Kovachki, N., Li, Z., Liu, B., Azizzadenesheli, K., Bhattacharya, K., Stuart, A., & Anandkumar, A. *Neural Operator: Learning Maps Between Function Spaces With Applications to PDEs*. Journal of Machine Learning Research, 23(241), 1–97, 2022.
10. Goswami, S., Lu, L., & Karniadakis, G.E. *Deep transfer operator learning for partial differential equations under conditional shift*. arXiv:2204.09810, 2022.

11. Xu, W., Lu, Y., & Wang, L. *Transfer Learning Enhanced DeepONet for Long-Time Prediction of Evolution Equations*. arXiv:2212.04663, 2022.

Приложение 1

Генерация данных для задачи вихря Тейлора–Грина

```
1 # --- Основной численный шаг -----
2 # u, v : скорости
3 # w_fourier : завихренность в спектральном пространстве
4 # f_fourier : внешнее воздействие
5 # dt, visc : шаг по времени и вязкость
6 # -----
7
8 # 1) Функция тока psi в спектре
9 psi_fourier = w_fourier / neg_laplacian_fourier
10
11 # 2) Скорости: u = dpsi/dy, v = -dpsi/dx
12 u_fourier = 2*math.pi*k_y*psi_fourier[...,1j]
13 v_fourier = -2*math.pi*k_x*psi_fourier[...,1j]
14
15 u = torch.irfft(u_fourier, 2, onesided=False,
16               signal_sizes=(N,N))
17 v = torch.irfft(v_fourier, 2, onesided=False,
18               signal_sizes=(N,N))
19
20 # 3) Градиенты вихревости
21 w_x = grad_x(w_fourier); w_y = grad_y(w_fourier)
22
23 # 4) Нелинейный член
24 nonlinear = torch.rfft(u*w_x + v*w_y, 2, onesided=False)
25
26 # 5) Шаг Крэнка-Николсона
27 denom = 1 + 0.5*dt*visc*neg_laplacian_fourier
28 w_fourier = (w_fourier + dt*(f_fourier - nonlinear)
29             - 0.5*dt*visc*neg_laplacian_fourier*w_fourier) /
30             denom
```

Листинг 1 – Генерация начального условия и численное решение уравнения Навье–Стокса

```
1 def taylor_green_ic(batch, Nx, amp_range=(1.0, 1.2),
2   add_noise=0.25):
```

```

2   t = torch.linspace(0, 1, Nx+1)[: -1]
3   X, Y = torch.meshgrid(t, t)
4
5   amplitude = 1
6   phase_x = 0
7   phase_y = 0
8
9   initial_vorticity_field = 2*math.pi * amplitude *
torch.sin(2*math.pi*(X+phase_x)) *
torch.sin(2*math.pi*(Y+phase_y))
10
11   if add_noise > 0:
12       random_noise = add_noise *
torch.randn_like(initial_vorticity_field)
13       initial_vorticity_field += random_noise -
random_noise.mean(dim=(-2,-1), keepdim=True)
14
15   return initial_vorticity_field

```

Листинг 2 – Генерация начального поля Тейлора–Грина

Приложение 2

DeepONet

```
1 def conv_block(x, filters, k=5, name=''):
2     """Conv2D → BatchNorm → swish + residual."""
3     x = tf.keras.layers.Conv2D(filters, k, padding='same',
4
5     kernel_initializer='glorot_normal',
6     use_bias=False,
7     name=name+'_conv')(x)
8     x = tf.keras.layers.BatchNormalization(name=name+'_bn')(x)
9     x = tf.keras.layers.Activation('swish', name=name+'_act')(x)
10    if x.shape[-1] == filters: #
11        identity-skip
12        skip = tf.keras.layers.Conv2D(filters, 1,
13        padding='same')(x)
14        x = tf.keras.layers.Add()([x, skip])
15    return x
```

Листинг 3 – Базовый сверточный блок с остаточным подключением

```
1 def get_B(self):
2     """CNNветвь- В обрабатывает( поле w_0)."""
3     inp = tf.keras.Input(shape=(self.H_input_size,)) #
4     vec → img
5     img = tf.keras.layers.Reshape((self.Nx, self.Nx, 1))(inp)
6
7     img = self.conv_block(img, 32, 5, 'b1')
8     img = self.conv_block(img, 64, 5, 'b2')
9     img = self.conv_block(img, 128, 5, 'b3')
10
11    flat = tf.keras.layers.Flatten()(img) #
12    → вектор признаков
13    U = tf.keras.layers.Dense(self.trunk_units, 'tanh')(flat)
14    V = tf.keras.layers.Dense(self.trunk_units, 'tanh')(flat)
15    h = tf.keras.layers.Dense(self.trunk_units, 'tanh')(flat)
16
17    for _ in range(self.trunk_layers - 1): #
18        masked MLP
```

```

16     a1 = h * U
17     a2 = (1 - h) * V
18     h = a1 + a2
19
20     out = tf.keras.layers.Dense(self.branch_basis_size *
21                                 self.trunk_basis_size,
22     'tanh')(h)
23     out = tf.keras.layers.Reshape((self.branch_basis_size *
24                                     self.trunk_basis_size,
25     1))(out)
26     out = tf.keras.layers.Conv1D(1, self.trunk_basis_size,
27                                   strides=self.trunk_basis_size,
28                                   use_bias=False)(out)
29     return tf.keras.Model(inp, out)

```

Листинг 4 – Ветвь *B*: извлечение глобальных коэффициентов из начального поля (остальные ветви устроены примерно также)

```

1 def compute_pde_loss(self, B_vec, x, y, w_true, psix_true,
2   psiy_true):
3     Слагаемое"" PINN: уравнения Пуассона + эволюция w.""
4     with tf.GradientTape(persistent=True) as tape:
5         tape.watch([x, y])
6         X = tf.concat([x, y], 1) # (N,2)
7
8         T_w = self.T_wvnn(X)
9         T_psi = self.T_psinn(X)
10        c = self.Bnn(B_vec)[: , :, 0]
11
12        w = tf.reduce_sum(c * T_w, 1, keepdims=True)
13        psi = tf.reduce_sum(c * T_psi, 1, keepdims=True)
14
15        w_x = tape.gradient(w, x); w_y = tape.gradient(w, y)
16        w_xx = tape.gradient(w_x, x); w_yy = tape.gradient(w_y,
17        y)
18        psi_x = tape.gradient(psi, x); psi_y =
19        tape.gradient(psi, y)

```



```

17     psi_xx = tape.gradient(psi_x, x); psi_yy =
    tape.gradient(psi_y, y)
18
19     res_poisson = psi_xx + psi_yy + w
20
21     lap_w = w_xx + w_yy
22     conv  = psi_y * w_x - psi_x * w_y
23     f_src = self.get_source_f(x, y)
24     res_evo = w + self.dt*(conv - self.nu*lap_w - f_src) -
    w_true
25
26     return tf.reduce_mean(res_poisson**2) +
    tf.reduce_mean(res_evo**2)

```

Листинг 5 – Physics-informed функционал потерь

Приложение 3

FNO

```
1 def compl_mul2d(a, b):
2     real = op(a[..., 0], b[..., 0]) - op(a[..., 1], b[..., 1])
3     imag = op(a[..., 1], b[..., 0]) + op(a[..., 0], b[..., 1])
4     return torch.stack([real, imag], dim=-1)
```

Листинг 6 – Комплексное умножение тензоров в 2-D спектре

```
1 class SpectralConv2d_fast(nn.Module):
2     def __init__(self, in_ch, out_ch, m1, m2):
3         super().__init__()
4         self.in_ch, self.out_ch = in_ch, out_ch
5         self.m1, self.m2 = m1, m2                # число мод
6                                                    # максимума N/2
7         scale = 1 / (in_ch * out_ch)
8         # отдельные веса для положительных и отрицательных  $k$ 
9         self.w1 = nn.Parameter(scale * torch.rand(in_ch,
10 out_ch, m1, m2, 2))
11         self.w2 = nn.Parameter(scale * torch.rand(in_ch,
12 out_ch, m1, m2, 2))
13
14     def forward(self, x):
15         B, _, H, W = x.shape
16         x = torch.fft.rfft2(x, norm='ortho')      #
17         → спектр
18         out = torch.zeros(B, self.in_ch, H, W//2+1, 2,
19                             device=x.device)
20         out[:, :, :self.m1, :self.m2] = compl_mul2d(x[:,
21 :, :self.m1, :self.m2], self.w1)
22         out[:, :, -self.m1:, :self.m2] = compl_mul2d(x[:,
23 :, -self.m1:, :self.m2], self.w2)
24         return torch.fft.irfft2(out, s=(H, W), norm='ortho')  #
25         ← физическое пространство
```

Листинг 7 – Быстрая спектральная свёртка (FNO-ядро)

```
1 class SimpleBlock2d(nn.Module):
```

```

2     def __init__(self, m, width, T_in):
3         super().__init__()
4         self.fc0 = nn.Linear(T_in + 2, width)          # + (x,y)
координаты
5         self.sconv = nn.ModuleList([SpectralConv2d_fast(width,
width, m, m) for _ in range(4)])
6         self.wskip = nn.ModuleList([nn.Conv1d(width, width, 1)
for _ in range(4)])
7         self.bn      = nn.ModuleList([nn.BatchNorm2d(width) for _
in range(4)])
8         self.fc1, self.fc2 = nn.Linear(width,128),
nn.Linear(128,1)
9
10        def forward(self, x):
11            B,H,W,_ = x.shape
12            x = self.fc0(x).permute(0,3,1,2)            # →
(B,C,H,W)
13            for k in range(4):
14                y = self.sconv[k](x)
15                z = self.wskip[k](x.view(B,-1,H*W)).view(B,-1,H,W)
16                x = F.relu(self.bn[k](y + z))
17            x = self.fc2(F.relu(self.fc1(x.permute(0,2,3,1))))
18            return x

```

Листинг 8 – Блок FNO с четырьмя спектральными слоями +
skip-Conv1D